

UAV Swarm Network Simulator for Emergency Communications



Universitat Oberta
de Catalunya

Pablo Ramírez Escudero

Telematics

Master's Degree in Telecommunications
Engineering

Name of the tutor:

José López Vicario

Name of the SRP:

Pere Tuset Peiró

June 2, 2025



This work is under the license Attribution-NonCommercial
<https://creativecommons.org/licenses/by-nc/3.0>

Final Work Card

Title of the work:	UAV Swarm Network Simulator for Emergency Communications
Name of the author:	Pablo Ramírez Escudero
Name of the tutor:	José López Vicario
Name of the SRP:	Pere Tuset Peiró
Date of delivery:	June 2, 2025
Studies or Program:	Master's Degree in Telecommunications Engineering
Area or the Final Work:	Telematics
Language of the work:	English
Keywords:	UAV networks, Swarming algorithms, Reinforcement learning, Deep Q-Network (DQN), NS-3 simulator, Multi-agent simulation

Abstract

Unmanned aerial vehicle (UAV) swarm networks offer a rapid solution to restore cellular communications in emergency scenarios, acting as airborne base stations when terrestrial infrastructure is not available. To maximise coverage beyond single UAV's range, drones form an autonomous ad-hoc for multi-hop data transmission. This work introduces two decentralised swarming algorithms to control autonomous UAV network formation. First, we propose a modified version of the original Extended Virtual Spring (EVSM) algorithm, including performance optimisations and improved collision avoidance mechanism. Next, we present the Swarm Deep Q-Network (SDQN) algorithm, which applies deep reinforcement learning to improve deployment efficiency by maximising user coverage while reducing the number of UAVs required. The SDQN agent is formulated within a multi-agent reinforcement learning (MARL) problem, proposing a difference reward scheme and log-polar image representation processed via a convolutional neural network (CNN) to achieve desired behaviour. Although EVSM demonstrates robust and consistent performance, SDQN did not achieve the expected results, and further improvements are left for future research. To evaluate swarming algorithms performance under realistic network conditions, we developed a hybrid simulation tool that combines a custom multi-agent dynamics simulator with the NS-3 network simulator through a socket-based protocol. This integrated tool provides a unique platform for researching UAV swarming techniques and wireless ad-hoc networks for autonomous aerial systems.

Contents

Abstract	iii
Table of Contents	iv
List of Figures	ix
List of Tables	x
1 Introduction	1
1.1 Project overview	1
1.2 Motivation	2
1.3 Objectives	3
1.4 Feasibility study	4
1.5 Ethical and global commitment and sustainable development goals	5
1.6 Project planning	6
2 State of the Art	9
2.1 UAV network applications	9
2.2 Swarm control paradigms	10
2.2.1 Formation control schemes	10
2.2.2 Conventional and AI-based strategies	10

2.3	Virtual forces approaches	11
2.3.1	Virtual spring mesh algorithm	11
2.3.2	Acute angle test	11
2.3.3	Extended virtual spring mesh	12
2.4	Machine learning for UAVs	13
2.4.1	Machine learning and deep learning	13
2.4.2	Reinforcement learning	15
2.4.3	Deep reinforcement learning	17
2.4.4	DQN for area coverage with UAVs	19
2.5	Communication technologies for UAV networks	20
2.5.1	Wireless ad-hoc networks	20
2.5.2	UAVs and cellular networks	21
2.6	UAV network simulation	21
3	System design	23
3.1	System architecture	23
3.2	Technology selection	24
3.2.1	UAV model selection	24
3.2.2	Communication standard selection	25
3.2.3	Communication range and number of UAVs	26
3.3	SwarmLink protocol	28
3.3.1	Position broadcast services	28
3.3.2	SwarmLink packet structure	28
4	Swarming algorithms	30
4.1	Modified EVSM algorithm	30

4.1.1	Potential collision avoidance force	31
4.1.2	Modified edge robot test	31
4.1.3	Improved acute angle test	32
4.1.4	Modified algorithm description	34
4.1.5	Position and altitude control	34
4.1.6	Performance validation	36
4.2	Swarm Deep Q-Network	37
4.2.1	SDQN agent	38
4.2.2	State representation	39
4.2.3	Reward scheme	42
4.2.4	Position and altitude control	44
4.2.5	Training process	44
5	Simulation application	46
5.1	Design process	46
5.1.1	System requirements	46
5.1.2	Preliminary design	47
5.1.3	Final architecture	49
5.2	Multi-agent simulator	49
5.2.1	Agents overview	50
5.2.2	Environment	50
5.2.3	Swarming algorithms	51
5.3	Network simulator	52
5.4	SimBridge protocol	53
5.4.1	Command codes	53
5.4.2	Packet formats	55

5.4.3	Interaction between simulators	55
5.4.4	NS-3 network simulator	56
5.4.5	Python multi-agent simulator	57
5.4.6	Simulators communication flow	57
5.5	Visualization applications	60
6	Simulation Results	63
6.1	Network simulator performance	63
6.2	SDQN training	65
6.3	Ideal vs network-based simulation	67
7	Conclusions	69
7.1	Project results	69
7.2	Future works	70
	Glossary	73
	Bibliography	73

List of Figures

1.1	Gantt diagram for project plan	6
2.1	VSM acute angle test	12
2.2	Machine learning categories diagram	14
2.3	Convolutional neural network for image classification	15
2.4	Agent-environment interaction in RL	16
3.1	Diagram of system agents interaction.	24
3.2	SwarmLink header format	29
3.3	SwarmLink position packet format	29
4.1	EVSM deployment test	37
4.2	Swarm DQN agent components interaction	38
4.3	Fixed-frame generator outputs from multi-agent simulator	40
4.4	Log-polar frame generator outputs	41
5.1	Simulator architectures diagrams	49
5.2	SDQN architecture diagram	52
5.3	SimBridge simulators interaction diagram	56
5.4	Communication flow between simulators with SimBridge protocol	59
5.5	EVSM simulator and network viewers	61

5.6	Mission area elevation and satellite images	62
5.7	PyVista 3D terrain visualizer	62
6.1	Network simulator test results	64
6.2	Training performance of the SDQN algorithm	67
6.3	EVSM coverage performance with ideal and network-based position data	68

List of Tables

1.1	Project tasks breakdown	7
3.1	Specifications of enterprise UAV models	25
3.2	Comparison of selected Wi-Fi modules for UAVs	25
4.1	CNN model of the DQN agent	39
5.1	Simulator components inputs and outputs	48
5.2	Mobility models and connection policies of each agent type	50
5.3	Bounding box coordinates for simulated scenario.	62
6.1	PC specifications for DQN training	65
6.2	Network simulator test results	65
6.3	SDQN Training Parameters	66
6.4	Time to reach full area and user coverage	68

Chapter 1

Introduction

This Master's final project proposes the development of an UAV swarm simulator to demonstrate the capability of autonomous swarm drones for re-establishing mobile communications in emergency scenarios. This work is focused on the research and development of decentralised swarming algorithms for UAV self-organisation and efficient coverage of the mission area.

1.1 Project overview

The proposed system consists of autonomous quadcopters acting as base stations of a mobile network. The autonomous drones will create a wireless ad-hoc network capable of transmitting data packets between swarm nodes. This network will provide connectivity for users on the ground, comprising members of rescue teams and victims, serving as a communications overhaul service when the cellular network infrastructure is not available.

The system is developed and tested within a simulation environment implemented in Python and integrated with the NS-3 network simulator. This will permit us not only to evaluate the decentralised swarming algorithms but also to perform a realistic simulation of wireless communication standards and the full Internet protocols stack. To facilitate visualisation of the simulation status and overall performance of the system, a graphical user interface will be implemented.

The software combines a self-developed UAV swarm simulation tool in Python with a network simulator scenario in NS-3 written in C++. The swarm simulator manages the simulation and control of three types of agents: drones, users, and control stations. The mission area is configured by defining a boundary and designating obstacles as avoidance zones. To achieve a realistic simulation, elevation data are incorporated to generate a 3D terrain surface. Drones will be deployed from the control station to cover the area within the boundary and provide wireless coverage to the users on the ground, while maintaining constant height to the terrain and avoiding collision with obstacles.

This research is focused on two swarming algorithms: Extended Virtual Spring Mesh (EVSM) and Swarming Deep Q-Network (SDQN). EVSM, as originally defined in [1], addresses the limitations of classical artificial force swarming methods. However, the EVSM implementation was revised after the original formulation was shown to be unstable in our simulation environment. In contrast, SDQN represents the application of reinforcement learning techniques [2] to UAV swarming, rather than defining a concrete algorithm. Existing studies employing Q-Learning for aerial base station placement have explored both centralised [3] and decentralised [4] approaches, demonstrating the effectiveness of reinforcement learning in optimising UAV positioning to maximise user coverage.

This work makes two key contributions. First, it provides a stable and revised implementation of EVSM. Second, it introduces a detailed formulation of the Swarm Deep Q-Network (SDQN) algorithm, incorporating multi-objective rewards to guide autonomous UAV deployment. Both algorithms were developed and tested considering communication restrictions within the NS-3 network simulator that models limited range, transmission delay, and packet loss, among other parameters. Although the SDQN algorithm did not achieve the expected performance, this research reveals key challenges in applying deep reinforcement learning to UAV swarm networks. The combination of advanced swarming algorithms and realistic network simulation provides a novel and practical contribution to the field of UAV swarming techniques.

1.2 Motivation

In recent years, significant research has been conducted on UAV-based systems due to their versatility and rapid deployment. In particular, swarm drones have demonstrated a wide range of applications, including serving as a safe alternative to fireworks at light shows, improving border surveillance, supporting search and rescue (SAR) missions, enabling precision agriculture, and assisting security forces operations such as military and police.

The tragic floods of 2024 in Spain exposed the limitations of emergency services delivering a rapid response to natural disasters. More recently, the widespread blackout on 28 April 2025 highlighted the vulnerability of critical infrastructure to large-scale disruptions. In such scenarios, UAV swarm networks offer an innovative solution for restoring mobile communications and assisting in the localisation of victims, improving the operational capacity of emergency teams. Drone swarms have the potential to replace the traditional deployment of ground-based stations, demonstrating a fast and effective alternative for re-establishing connectivity in crisis scenarios.

This project is relevant in the context of the Master's Degree in Telecommunication Engineering for studying the use of UAV swarms to create wireless networks and enhance the capabilities of mobile communications. This is a novelty area in the telecommunications field and is especially important for 5G and other new-generation networks [5].

The development of this simulator will allow us to apply advance mobile communication concepts such as dynamic ad-hoc routing, as well as machine learning techniques for network planning and optimisation. In particular, it will show the novel use of artificial intelligence and

reinforcement learning algorithms to coordinate and control drone swarms, allowing drones to solve multiple tasks in complex scenarios. In addition, there are few applications that integrate drones and network simulation, such as FlyNetSim [6], making this work a unique tool for research on wireless networking and swarming techniques for autonomous aerial systems.

1.3 Objectives

The main objective of this project is to demonstrate the capability of drone swarms to rapidly deploy and re-establish mobile communications, serving as an overhaul system for cellular networks. The work comprises the development of the simulation tool, the research and development of decentralised swarming algorithms, and the analysis and evaluation of the simulation results.

The main technical objectives or high-level requirements for this project are the following:

- **Connectivity between users through the simulated network:** This will prove the correct operation and integration of the modules implemented within the simulator. The exchange of data packets between users will only be possible if drones are properly deployed, wireless connections are established, and packets are routed through the ad-hoc network.
- **Feasibility of UAV swarm for overhaul communications:** This will be evaluated by analysing network performance metrics. Key QoS indicators such as latency, throughput, and packet loss rate will be assessed to determine the reliability and efficiency of wireless communications and ad-hoc routing.
- **Optimal area coverage using decentralised swarming algorithms:** Swarm drones shall self-organise to achieve a spatial distribution to efficiently cover the entire area of the mission, while avoiding collisions with other neighbour drones and keeping their positions within the boundary limits. This will be verified using the metrics described in [1], such as area coverage effectiveness, fraction of drones connected to the swarm.
- **Development of a fully integrated UAV swarm network simulator:** The final product of this project will be a functional application, with a simple installation and usage process. The internal modules must be tested and verified. The application shall include all necessary packages and containerisation technologies such as Docker might be used to ensure the compatibility of the software across different operating systems.

The secondary objectives of the project are as follows:

- **Interoperability between NS-3 and the Python-based simulator:** Verify the correct integration and communication using a simple socket-based API. This also involves developing control mechanisms to manage data flow, ensure synchronisation, and handle potential errors.
- **Real-time simulation:** To simulate the system in real-time the Python code will be optimised using highly vectorised operations with NumPy and TensorFlow, and JIT compiled

functions with Numba. Depending on the number of agents simulated, the complexity of the algorithms employed, and the available computational resources, it may be necessary to run the simulation in 'slow time'.

- **Modularity and scalability:** To design a modular architecture that facilitates the integration of future extensions and improvements, ensuring the scalability of the system. Although this objective is more difficult to evaluate, it can be checked if performance is affected by the addition of new modules or an increase in the number of simulated agents.

1.4 Feasibility study

The deployment of UAV networks to restore communications in emergency situations has been widely studied, as explained in Chapter 2, where numerous techniques are presented to address the challenges of such scenarios. Furthermore, the technologies selected for the development of the simulator are mature, although they present some implementation challenges, they do not mean a significant obstacle to the success of the project.

Several potential risks have been identified along with mitigation measures. These risks are classified into three categories: aspects related to simulation software, limitations concerning the realism of the models, and the viability of the designed system in real-world environments.

Risks related to the simulator:

- **Processing capacity:** Simulating up to 100 agents, each running a machine learning algorithm, at the same time with a complete simulation of the network protocol stack in NS-3, could be very demanding in terms of processing power. Depending on the available computing resources, the proposed real-time simulation might be degraded to a "slow-time" execution. To mitigate this, the code will be optimised through efficient algorithm implementations, JIT (Just-In-Time) compilation, and parallel processing methods. Moreover, the proper installation of TensorFlow with CUDA drivers will be essential to leverage GPU acceleration of IA models.
- **Integration with NS-3:** The interconnection between the multi-agent simulator and NS-3 could be complex and will require a control mechanism to ensure the correct communication and time synchronisation of both simulation systems. The inter-processes communication approach followed in FlyNetSim [6] will be adopted to ensure the success of integration. Further details of integration with NS-3 are discussed in Section 5.4.

Limitations in model realism:

- **Simplified UAV model:** Aspects such as UAV aerodynamics, flight mechanics, autopilot behaviour, or sensor noise characteristics are not considered in this simulator. These aspects are beyond the scope of this project and may limit the application of simulation results to real-world scenarios. A modular code structure will be adopted during the implementation of the software, in order to facilitate future extensions for more realistic physical simulation and control modules.

- **Wireless communications modelling:** The channel model used in NS-3 will be a simple line-of-sight (LOS) model, potentially reducing precision in scenarios involving shadowing, multipath propagation, and interference. The implementation of more realistic air-to-ground channel models would be a complex task, as analysed in [7] and [8], and will not be addressed in this project.

Viability of the system in real-world scenarios:

- **Aerial vehicle class:** The high mobility of quadrotors simplifies the design of positioning algorithms. However, their limited battery lifetime presents significant challenges and makes them not suitable for long-duration missions. Vehicles with a longer operating range, such as fixed-wing UAVs and high-altitude platforms (HAPs), may be more appropriate for this application, but these types of vehicle require a different kind of algorithms related to path planning. As explained in [7], small UAVs reduce the operational cost of the system and will therefore be chosen for these projects.
- **Flight altitude regulations:** According to [9], the optimal flight altitude to ensure adequate coverage and reduce inter-drone interferences is approximately 1000 metres, when operating 10 or more UAVs. However, in countries such as Spain, current regulations prohibit the operation of drones above 50 metres. This legal restriction may prevent the practical feasibility of the system until regulatory changes are made.

1.5 Ethical and global commitment and sustainable development goals

The increasing use of autonomous robots and UAV swarms in military operations and armed conflict, such as the ongoing war in Ukraine, raises ethical concerns about these technologies. In contrast, the present Master's Final Project proposes the use of UAVs in civil applications, such as enhancing current mobile infrastructure to make it more resilient and reliable, as well as supporting emergency operations in disaster scenarios. These applications align with a strong ethical and global commitment to improve quality of life and potentially save human lives.

In relation to the Sustainable Development Goals (SDG) of the United Nations [10], this project includes several key aspects related to clean energy usage and environmental sustainability.

The ground control stations responsible for UAV deployment and battery recharging can be powered by renewable energy sources, such as small-scale wind turbines or photovoltaic panels. This enables the design of a zero-emission system that aligns with SDG 9 (Industry, Innovation, and Infrastructure). However, in emergency scenarios, sustainability is not the main focus, but availability. This makes fuel-powered generators a more viable and immediate energy source. This trade-off must be considered to keep the design aligned with SDG 13 (Climate Action).

Another environmental aspect to consider is the sustainability of the materials used in the construction of UAVs. Small-scale UAVs (typically under 5 kg) are generally powered by lithium polymer (Li-Po) or lithium-ion batteries (Li-Ion). These types of batteries are difficult

to recycle and pose a significant environmental hazard [11]. When the use of these materials cannot be avoided, appropriate disposal must be considered to remain consistent with the objectives of SDG 13.

In addition, this project promotes energy efficiency through the optimisation of UAVs positions to maximise user coverage. Efficient placement of aerial base stations not only improves mission performance, but also reduces the number of UAVs required, thereby reducing energy consumption and environmental impact, reinforcing the project’s alignment with SDG 13 (Climate Action).

1.6 Project planning

On 2 June 2025, the current project plan status is as follows:

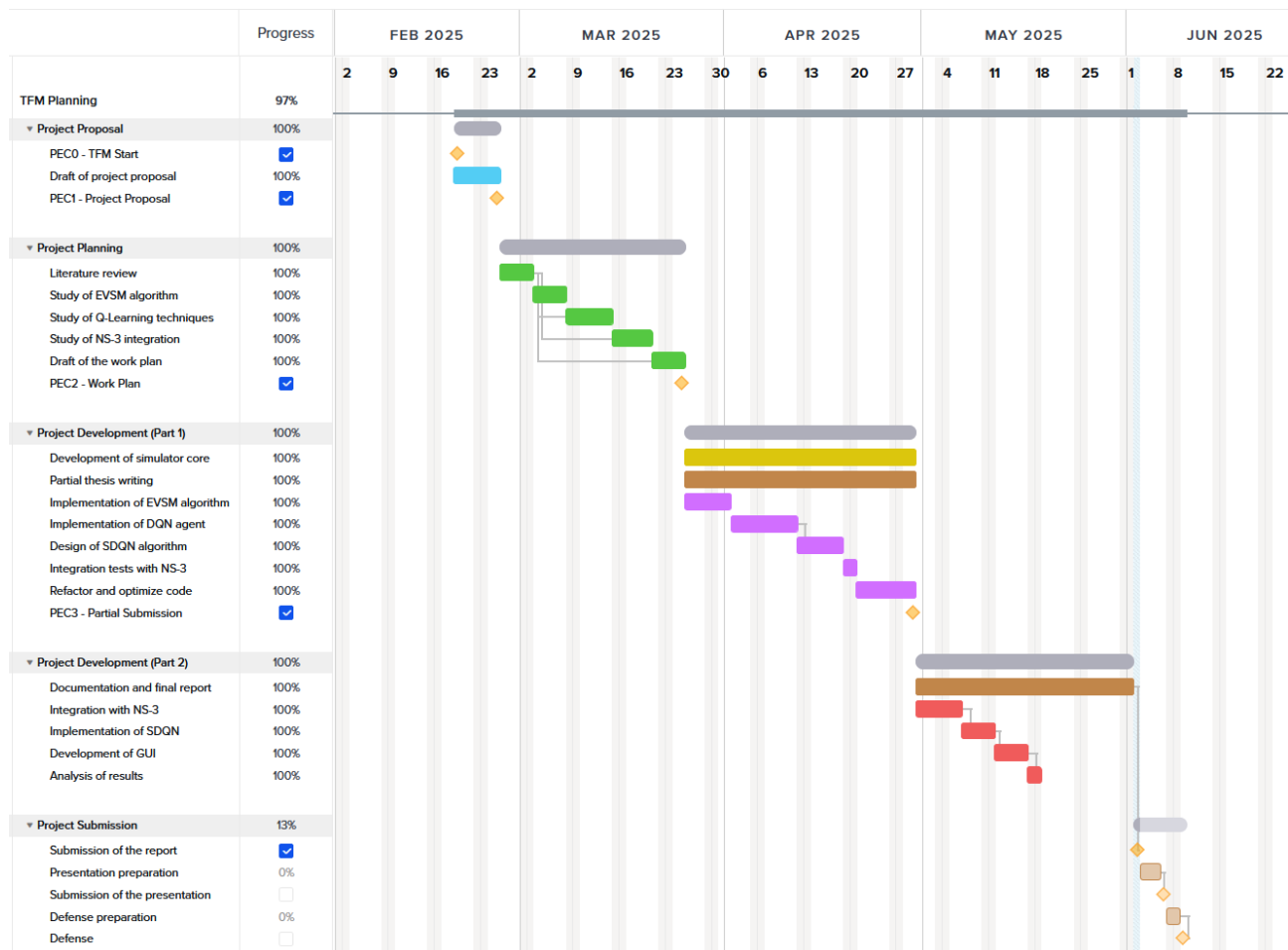


Figure 1.1: Gantt diagram for project plan

The following table provides the project tasks breakdown and description:

Table 1.1: Project tasks breakdown

Task	Start	End	Description
Project Proposal			
PEC0 – TFM Start	19-Feb	19-Feb	Beginning of the Master’s Final Project.
Draft of project proposal	19-Feb	25-Feb	Development of the project proposal document.
PEC1 – Project Proposal	25-Feb	25-Feb	Submission of project proposal document.
Project Planning			
Literature review	26-Feb	02-Mar	Search and review of relevant scientific articles.
Study of EVSM algorithm	03-Mar	07-Mar	Research on the EVSM algorithm and development of a small test program.
Study of Q-Learning techniques	08-Mar	14-Mar	Review of machine learning theory, implementation of DQN agents and development of a test program.
Study of NS-3 integration with Python	15-Mar	20-Mar	Analysis of integrating the NS-3 simulator with Python, implementation of sockets, and study of NS-3 models.
Draft of the work plan	21-Mar	25-Mar	Development of work plan document.
PEC2 – Work Plan	25-Mar	25-Mar	Submission of work plan document.
Project Development (Part 1)			
Development of simulator core	26-Mar	29-Apr	Implementation of the general simulator code to integrate the other components and modules.
gorithm	26-Mar	01-Apr	Development of custom EVSM algorithm implementation in Python code.
Implementation of DQN agents	02-Apr	11-Apr	Development of hand coded DQN library in Python and TensorFlow. Implement small testing scripts.
Design of SDQN swarming algorithm	12-Apr	18-Apr	Design of a custom DQN-based swarming algorithm over existing simulation code and implemented DQN library.
Integration tests with NS-3	19-Apr	20-Apr	Development of test code to validate communication between Python and NS-3 through sockets.
Code optimization and refactoring	21-Apr	29-Apr	Optimization of simulator code to improve performance.
PEC3 – Partial Submission	29-Apr	29-Apr	Submission of the current state of the thesis writing.
Project Development (Part 2)			
Documentation and final report	30-Apr	01-Jun	Finalization of the project report writing.
Integration with NS-3	30-Apr	06-May	Development of the library to interconnect the Python multi-agent simulator with NS-3 using sockets.

Continued on next page

Task	Start	End	Description
Implementation of SDQN algorithm	07-May	11-May	Development of the final version of DQN-based swarming algorithm.
Development of the graphical interface	12-May	16-May	Creation of the GUI to visualize the simulation state and the control application.
Analysis of simulation results	17-May	18-May	Comparison of results and generation of graphs and other data visualization tools.
Project Submission			
Submission of the thesis	02-Jun	02-Jun	Submission of the final version of the thesis writing.
Presentation preparation	03-Jun	05-Jun	Creation of the presentation slides to summarise the key points of the project.
Submission of the presentation	06-Jun	06-Jun	Submission of the presentation slides.
Defence preparation	07-Jun	08-Jun	Preparation of presentation speech and potential questions from the evaluation committee.
Defence	09-Jun	09-Jun	Online defence of the Master's thesis and project.

Chapter 2

State of the Art

2.1 UAV network applications

Drones, also known as unmanned aerial vehicles (UAVs), have gained popularity in civil applications due to their simple deployment, low cost, and high mobility. In the 2019 systematic review [9], the authors identified the massive use of smart UAVs as one of the next major technological revolutions, projecting a market value of 45 billion dollars derived from their application in civil infrastructure.

The main UAV applications identified in [9] are package delivery, precision agriculture, surveillance, search and rescue (SAR) operations, and improvement of wireless coverage. This Master's Final Project focusses on the last application, specifically the use of UAVs to rapidly restore mobile communications after disasters. The authors also highlight similar applications with more commercial focus, such as the deployment of UAVs to enhance the existing mobile infrastructure.

In contrast to these applications, several challenges associated with the use of UAVs have been identified. These include battery limitations and autonomous charging, swarm management and obstacle avoidance, ad-hoc network routing difficulties, and other security-related problems [9].

Between 2015 and 2020, research focused on applying UAV networks to different fields, modelling their characteristics, and conducting simulation-based performance evaluations. In the 2015 article [12], the use of UAVs as unmanned aerial bases (UABS) was proposed to restore mobile broadband services, working with terrestrial base stations to form a heterogeneous network. Simulations demonstrated the potential of UAVs to maximise coverage and throughput in damaged infrastructure. The 2018 review [7] examined the role of UAVs in wireless networks, classifying platforms by operational altitude (High-Altitude Platforms, HAPs and Low-Altitude Platforms, LAPs) and flight characteristics (fixed-wing and rotary-wing), and addressing the challenges of modelling the air-to-ground (A2G) communication channel and optimally deploying UAVs as base stations. The article [13] provides a comprehensive analysis of network architectures, access and routing protocols, and the main applications based on the characteristics of flying ad hoc networks (FANET).

2.2 Swarm control paradigms

Drone swarms have many advantages over operating single UAVs such as resilience, scalability, and operational cost. The problem of formation control to enable swarm drones to navigate autonomously in dynamic environments and fulfil complex missions have been widely studied. The 2024 review [14] provides an in-depth study of current swarm control paradigms, analysing conventional control strategies and novel AI-based approaches.

2.2.1 Formation control schemes

The formation control of UAV swarms can be classified into centralised, decentralised, and distributed schemes [14].

- **Centralised:** In the centralised scheme, all UAVs are commanded from a single control station. Its main advantages are its simple decision-making process and consistency in the control commands. However, it suffers from scalability issues, single point failure, and communication overhead.
- **Decentralised:** In the decentralised scheme, UAVs act based on predefined rules and available information, coordinating with their near neighbours to maintain swarm formation. Advantages of this method are the improved scalability of not depending on a central control station, and the high adaptability to react to changes in its close environment. Despite this, its main challenges are coordination and more complex design of control laws.
- **Distributed:** The distributed scheme is similar to the decentralised approach but with UAVs collaborating with other neighbours by sharing information and making collective decisions. This method makes intensive use of communications and requires a more complex decision-making process to reach consensus.

2.2.2 Conventional and AI-based strategies

Drone swarms control methods can be divided into conventional and AI-based [14]. Conventional strategies rely on hand-designed algorithms with predefined rules, predictable behaviour, and proved stability. Meanwhile, AI-based methods are based on heuristic algorithms and deep learning techniques that automatically learn coordination policies from the data, improving adaptability and performance in complex and challenging tasks.

Conventional strategies include virtual structure formation, leader-follower methods, behaviour-based control, and physics-based control [14].

- In virtual structure formation, each UAV is assigned a specific position to create a geometric pattern.
- The leader-follower method is a hierarchical approach in which one or several UAVs are designated as leaders and each follower only focusses on its near leader, simplifying control logic, and reducing the coordination complexity.
- In behaviour-based control, each UAV makes decisions based on available information of local

environment and neighbour UAVs. This decentralised approach is inspired by the flocking behaviour of birds.

- Physics-based control simulates artificial repulsive and attractive forces to control UAV positions, converging to specific geometry formations. Some approaches use a force field analogous to gravity or electromagnetic forces, while others simulate virtual springs between drones [15].

AI-based approaches use artificial neural networks (ANNs) and deep reinforcement learning (DRL) to adapt to complex scenarios [14]. These techniques include value-based methods such as deep Q-network (DQN), an extension of the classical Q-learning algorithm using deep learning; policy-based techniques such as the policy gradient method; and hybrid approaches such as actor-critic algorithms, which combine learning a policy (the actor) and a value function (the critic) simultaneously.

For this project, two control strategies are selected: a conventional control method based on virtual physics, known as the EVSM algorithm [1], and a value-based DRL approach using the DQN algorithm [16], which we refer to as Swarm Deep Q-Network (SDQN). The theoretical background of these swarming algorithms is studied in the following sections.

2.3 Virtual forces approaches

2.3.1 Virtual spring mesh algorithm

Virtual Spring Mesh (VSM), introduced in [15], proposes a simple position control mechanism based on virtual forces to maintain uniform spacing between UAVs. VSM simulates virtual springs between neighbour drones, generating a repulsion force when two drones are too close and an attraction force when drones are too far away. The control law for each robot is

$$\ddot{\mathbf{x}} = \left[\sum_i k_s (l_i - l_0) \hat{\mathbf{u}}_i \right] - k_d \dot{\mathbf{x}} \quad (2.1)$$

where $\ddot{\mathbf{x}}$ is the drone acceleration, $\dot{\mathbf{x}}$ is the drone's velocity, l_i is the distance to the i -th connected neighbour, and $\hat{\mathbf{u}}_i$ is the direction to the i -th neighbour. The control parameters are natural length l_0 , spring constant k_s , and damping coefficient k_d . This control law can be treated as a control force by multiplying the acceleration of the drone by its mass: $\mathbf{F}_{\text{ctrl}} = m \cdot \ddot{\mathbf{x}}$.

The damping force in Equation 2.1 is added to ensure convergence to static positions by applying a decelerating force proportional to the speed of the drone and opposite to the direction of movement. This converts the energy-conservative spring systems into a non-conservative mass-spring-damper system with dissipation, enabling convergence to static equilibrium.

2.3.2 Acute angle test

The performance of the virtual spring mesh depends on the topology of the mesh. There are several approaches to mesh formation with different geometric properties and computational complexity. A full connectivity mesh is the simplest algorithm but exhibits severe instability and suffers spring

stacking, causing edge drones to compress against the centre. Another simple method will be to form connections with the N nearest neighbours or with drones within a given distance. However, these methods can cause neighbours to disperse and do not solve the spring stacking problem.

The method proposed in [15] is the acute angle test. This algorithm will form a spring between a drone A and drone B if for all other neighbours C_i the inscribed angle $\angle AC_iB$ is acute. The algorithm can be implemented by checking whether any neighbour C_i is within the circle for which A and B form its diameter. The Figure 2.1 illustrates this concept.

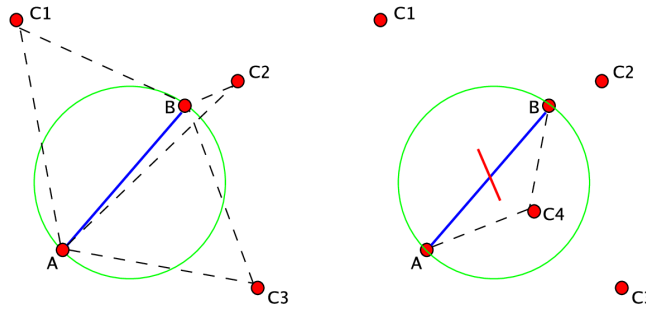


Figure 2.1: Satisfied acute angle test (right) and unsatisfied (left), taken from [15].

The acute angle test provides a fully connected planar graph, regardless of the distribution of drones. This topology formation method, combined with the virtual spring forces, allows the swarm to converge to a hexagonal uniform mesh in which all internal drones are connected to 6 neighbours, except in the case of edge drones.

2.3.3 Extended virtual spring mesh

The extended virtual spring mesh (EVSM) was first introduced in [1] and reviewed in [17] as an improvement to the virtual spring mesh (VSM) algorithm, promoting exploration of the environment and avoiding collisions with obstacles. This improvement behaviour is achieved by adding new exploration and expansion forces, a collision avoidance mechanism, and adaptative spring length.

The exploration control law defines a force that moves the edge robots in the direction of distant obstacles, pulling the interior robots along. The exploration force for each drone is

$$\mathbf{F}_{\text{expl}} = \left[\sum_i (m \cdot \omega_i) k_s (l_i - l_0) \hat{\mathbf{u}}_i \right] - k_d \dot{\mathbf{x}} \quad (2.2)$$

where m is a force multiplier, ω_i is a normalised weight related to the i -th obstacle, and $\hat{\mathbf{u}}_i$ is the direction to the i -th obstacle.

Visible walls or obstacles within the sweep angle of an edge drone are assigned weights proportional to their relative distances, making far obstacles more attractive than the closest ones. The sweep angle is defined as the largest angle of an edge drone in which there are no visible neighbour drones. The weight for the obstacle i is calculated as

$$\omega_i = \frac{d_i}{\sqrt{\sum d_i^2}} \quad (2.3)$$

where d_i is the distance to the closest point of the i -th visible obstacle.

Each robot in the formation is determined to be an interior or an edge. In the original algorithm of [1] a robot in position p_i must satisfy two conditions to be considered an edge robot:

1. Two neighbours at positions p_j and p_k must be seen with an angle greater than $\theta_i \geq 90^\circ$.
2. There must be no neighbours inside the polygon formed with p_i , p_j , p_k , and the point p_m projected from p_i at an angle $\theta_i/2$ and distance d_m .

In [1], collision avoidance is implemented by flipping the sign of the total force $\mathbf{F}_{\text{total}} = \mathbf{F}_{\text{ctrl}} + \mathbf{F}_{\text{expl}}$, where $\mathbf{F}_{\text{ctrl}}$ and $\mathbf{F}_{\text{expl}}$ are defined in Equations 2.1 and 2.2, respectively. When the drone's distance to the nearest obstacle falls below a threshold $d_{\min}$, the total force is reversed: $\mathbf{F}_{\text{total}} \rightarrow -\mathbf{F}_{\text{total}}$. However, this approach has not worked in testing simulations and was redesigned in our custom implementation of the EVSM algorithm described in Section 4.1.

2.4 Machine learning for UAVs

Recent studies have explored the application of Artificial Intelligence [8] and Reinforcement Learning [2] to address the challenges of UAV networks, trying to provide innovative solutions in complex scenarios. The 2023 review [8] compiles and analyses numerous research on AI algorithms designed to optimise UAV deployment and energy consumption, intelligent routing and networking techniques for ad-hoc networks, and AI-powered network security mechanisms, among others. A more recent survey in 2025 [2] summarises the application of Deep Reinforcement Learning (DRL) techniques to solve the complex optimisation problems of UAV networks, classifying algorithms into four categories based on their application to different types of challenges.

2.4.1 Machine learning and deep learning

Machine learning (ML) is a field of study in artificial intelligence that provides methods to detect patterns in data and perform tasks autonomously. There are three primary categories of machine learning tasks [18]:

- **Supervised learning** uses labelled datasets to train algorithms to classify data or predict outcomes accurately.
- **Unsupervised learning** is used to analyse and group unlabelled datasets by discovering hidden patterns in the data.
- **Reinforcement learning** allows artificial agents to learn from the environment through trial and error, learning the best sequence of actions to take to maximise cumulative rewards.

Deep learning (DL) is a subset of machine learning (ML) that uses multilayered artificial neural networks (ANNs) to solve complex, non-linear and high-dimensional problems. These deep neural networks (DNNs) are inspired by the structure of the human brain [20]. In contrast to traditional ML methods that depend on hand-crafted features, deep learning models can process raw data, such as images or text, to automatically discover hidden patterns and extract relevant features.

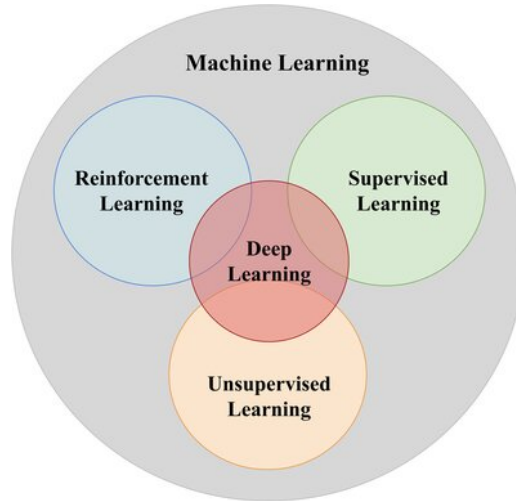


Figure 2.2: The three categories of machine learning, taken from [19].

Deep neural networks

A DNN consists of a sequence of multiple processing layers applying non-linear transformations that leads to learning different levels of abstraction. In a DNN, each neurone in the layer receives its input from all neurones in the previous layer, forming a fully connected or dense architecture. The simplest network, known as the perceptron, comprises a single artificial neurone. The perceptron take an array of inputs x_i , computes a weighted sum, and applies an activation function to the result. By convention, the first input $x_0 = 1$ is used to implement the bias term $b = w_0$. The output from a single perceptron can be defined as

$$y = A \left(\sum_{i=0}^N w_i \cdot x_i \right) = A \left(b + \sum_{i=1}^N w_i \cdot x_i \right) \quad (2.4)$$

where $A(\cdot)$ is the activation function. Common implementations employ non-linear activation function such as sigmoid, hyperbolic tangent or ReLU.

Training a DNN implies adjusting every weight and bias in order to minimise a loss function that measures the difference between network prediction and expected output. The most common method to optimise these parameters is the gradient descent method with the backpropagation algorithm [21]. The objective is to find the internal parameters θ that minimises the loss function $\mathcal{L}(\theta)$:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} \mathcal{L}(\theta) \quad (2.5)$$

where α is the learning rate and $\nabla_{\theta} \mathcal{L}(\theta)$ is the gradient of the loss with respect to the parameters. The loss function measures how well or poorly the neural network's predictions match the expected outputs.

Convolutional neural networks

Convolutional neural networks (CNNs) are a specialised form of ANNs particularly suitable for grid-structured data such as images. Instead of fully connected layers, as in DNNs, CNNs consist of a set of

learnable filters, also called kernels, which apply convolutional operations to the input. The network learns filters that activate when specific features are detected. First convolution layers learn how to detect edges, textures, and other simple patterns, while the following layers learn to detect parts and whole objects. In image classification problems, several convolutional layers are usually stacked and connected to fully connected layers to produce the final class scores.

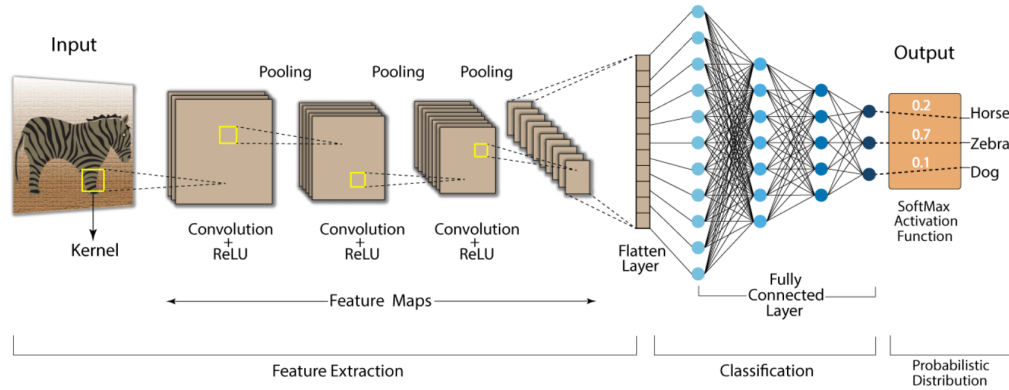


Figure 2.3: Convolutional neural network for image classification

While DNNs and CNNs are the foundation of deep learning, other architectures such as recurrent neural networks (RNN) and Transformers have demonstrated remarkable success in the natural language processing and content generation, ChatGPT or SORA are examples of these applications. In Section 2.4.3, we explain the application of deep neural networks to reinforcement learning to leverage the performance of traditional RL algorithms.

2.4.2 Reinforcement learning

The key aspect of RL is that the agent uses trial-and-error experience to learn “good behaviour”. The agent does not require complete knowledge or control of the environment, just to be able to interact with the environment through actions and collect information as current state and received rewards [22]. The agent can explore the environment randomly to learn from new experiences or use the learnt knowledge to take the best action that improves performance. How to balance random exploration with exploitation of learnt behaviour is one of the most important challenges on RL and it is called the exploration-exploitation dilemma [23].

Markov decision process

In reinforcement learning, an agent interacts with its environment over discrete time steps t , observing a state ω_t , taking an action a_t , and receiving a reward r_t and a new state observation ω_{t+1} in return (see Figure 2.4). This interaction is formalised as a discrete time stochastic control process known as Markov Decision Process (MDP) [24], a mathematical framework defined by a tuple (S, A, T, R, γ) :

- S is the state space
- A is the action state

- T is the transition function, defined as $T(s_t, a_t, s_{t+1}) : S \times A \times S \rightarrow [0, 1]$, it provides the probability of moving from a state s_t to a state s_{t+1} by taking action a_t .
- R is the reward function, defined as $R(s_t, a_t) : S \times A \rightarrow \mathbb{R}$, it measures the immediate reward of taking action a_t at state s_t .
- $\gamma \in [0, 1)$ is the discount factor.

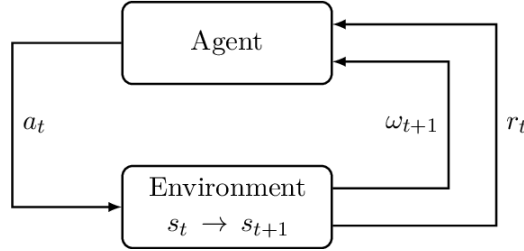


Figure 2.4: Interaction between agent and environment in RL, taken from [22].

In the context of RL, the agent interacts with the environment following a policy π . Policies can be classified as deterministic $\pi(s) : S \rightarrow A$, where the policy provides a single action a for each state s , or stochastic $\pi(s, a) : S \times A \rightarrow [0, 1]$, where policy defines the probability of selecting an action a in a state s . The optimisation problem is formulated through the reward scheme, where the agent learns the policy that maximises cumulative rewards [22].

Evaluating the expected performance of a fixed policy involves calculating the value functions. The state value function $V^\pi(s) : S \rightarrow \mathbb{R}$ defines the expected cumulative reward when starting from state s and following a policy π . Furthermore, the action-state value function $Q^\pi(s, a) : S \times A \rightarrow \mathbb{R}$ measure the impact of a specific action π . These functions are defined as follows [22]:

$$V^\pi(s) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s, \pi \right] \quad (2.6)$$

$$Q^\pi(s, a) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s, a_t = a, \pi \right] \quad (2.7)$$

where $r_t = \mathbb{E}[R(s_t, a_t, s_{t+1})]$ and $\mathbb{P}(s_{t+1} | s_t, a_t) = T(s_t, a_t, s_{t+1})$.

Bellman equations

In an MDP, these functions can be rewritten recursively using Bellman equations, establishing a recursive relation between V^π and Q^π for any state s_t and action a_t [24]. This relation is expressed in the following equations:

$$V^\pi(s_t) = Q^\pi(s_t, a_t) \quad (2.8)$$

$$Q^\pi(s_t, a_t) = R(s_t, a_t) + \gamma \sum_{s_{t+1} \in S} T(s_t, a_t, s_{t+1}) V^\pi(s_{t+1}) \quad (2.9)$$

where action a_t is provided by policy $\pi(s_t) = a_t$.

The optimal policy π^* maximises the value function V^π and action-value function Q^π for all states s_t and actions a_t . The relation of these optimal value functions is described by Bellman optimality equations:

$$V^*(s_t) = \max_{a \in A} Q^{\pi^*}(s_t, a) \quad (2.10)$$

$$Q^*(s_t, a_t) = R(s_t, a_t) + \gamma \sum_{s_{t+1} \in S} T(s_t, a_t, s_{t+1}) V^*(s_{t+1}) \quad (2.11)$$

where a represents all possible actions from the actions space A .

The optimal policy π^* can then be derived from the optimal action-value function Q^{π^*} through greedy action:

$$\pi^*(s_t) = \arg \max_{a \in A} Q^{\pi^*}(s_t, a) \quad (2.12)$$

There are three main families of RL algorithms: value-based, policy gradient, and model-based methods. The techniques studied in this work are value-based methods. This class of algorithms learns a value function ($V(s)$ or $Q(s, a)$) and then derives a policy from it.

Q-Learning

The simplest and most popular value-based method is the Q-Learning algorithm, which keeps a lookup table of values $Q(s, a)$ with one entry for every state-action pair [22]. In order to learn the Q-table, this algorithm makes use of the following recursive expressions:

$$Q_{\theta^-}(s_t, a_t) = R(s_t, a_t) + \gamma \max_{a \in A} Q_{\theta}(s_{t+1}, a) \quad (2.13)$$

$$Q_{\theta}(s_t, a_t) \leftarrow (1 - \alpha) Q_{\theta}(s_t, a_t) + \alpha Q_{\theta^-}(s_t, a_t) \quad (2.14)$$

where $\alpha \in [0, 1]$ is the learning rate, $\gamma \in [0, 1]$ is the discount factor, Q_{θ} represents the current estimated action-value function, and Q_{θ^-} is the target function estimated with Bellman equation (θ^- denotes a delayed copy of table parameters θ).

To learn effectively, Q-Learning must balance exploration with exploitation. A common strategy is an ε -greedy policy, where the agent chooses a random action with probability ε , and the optimal action with probability $1 - \varepsilon$. Typical implementations of Q-Learning start with a ε value of 1 to maximise exploration and progressively reduce its value through training steps to introduce exploitation.

Q-Learning is often inapplicable with high-dimensional or continue state-action spaces, which require to define a matrix of enormous size to store each state-action pair entry. In 2015, Google's DeepMind team introduced the DQN (Deep Q-Network) algorithm [25] to solve these limitations of Q-Learning, applying deep neural networks (DNN) to approximate the Q function by learning from high-dimensional data.

2.4.3 Deep reinforcement learning

Deep reinforcement learning (DRL) is the combination of reinforcement learning (RL) and deep learning (DL). This field has been able to solve complex tasks that were previously beyond the reach of classic machine learning algorithms.

Deep Q-Network

One of the most popular DRL algorithms is Deep Q-Learning, also known as Deep Q-Network (DQN) [16]. It uses convolutional neural networks (CNNs) for feature representation of high-dimensional input data such as images, enabling advanced performance in complex game environments. DQN introduces several key techniques to improve algorithm performance and stabilise the training process.

A commonly used technique in DQN is experience replay memory [26]. This approach combines online and offline learning by storing the last N experiences collected from the environment following a ε -greedy policy. Periodically, the DQN agent performs a train step by taking a random set of experience tuples $\langle s_k, a_k, r_k, s_{k+1} \rangle$, called mini-batch, from the replay buffer. This breaks the correlation between consecutive experiences, allowing the agent to learn an optimal policy for all stages in the game.

In DQN, the network parameters are updated to minimise the square loss function, which measures the difference between predicted Q-values and the target Q-values. For an experience $\langle s_k, a_k, r_k, s_{k+1} \rangle$, the target value is defined as

$$y_k = r_k + \gamma \max_{a \in A} Q_{\theta^-}(s_{k+1}, a), \quad (2.15)$$

where Q_{θ^-} denotes the target network and γ is the discount factor. The Q-network is updated by minimizing the mean squared error (MSE) between the predicted Q-value $Q_{\theta}(s_k, a_k)$ and the target value y_k , calculated as

$$\mathcal{L}(\theta) = (y_k - Q_{\theta}(s_k, a_k))^2. \quad (2.16)$$

This optimization is performed using stochastic gradient descent and backpropagation [22].

Double Deep Q-Network

Another relevant technique to improve DQN is double Q-learning [27]. Classic Q-Learning tends to overestimate the value of future rewards due to the max operator, causing instabilities during the training process and increasing the time required to achieve a certain level of success. Double Deep Q-Network (DDQN) [28] addresses this by decoupling the selection and evaluation of actions using two separate neural networks. One network, referred to as the Q-target, is used to estimate the value of future actions. The target network is updated less frequently, by copying the weights of the main Q-Network, to stabilise the learning process. The setup described in [16] typically updates the Q-target network every 10,000 training steps.

Prioritised experiences replay

Methods based on DQN typically use a replay memory to stabilise the learning process, by sampling stored experiences uniformly to form a mini-batch for training. However, not all experiences are equally informative. Prioritised Experience Replay (PER), introduced in [29], improves learning and reduces training time by sampling more frequency transitions with a higher temporal difference (TD) error, allowing the agent to focus on learning from unexpected experiences.

The TD error δ is defined as the absolute difference between the expected reward r'_t predicted by the Q network and the current reward r_t obtained from the environment. The PER method avoids the overfitting issues of greedy prioritisation of TD errors, using a stochastic sampling method that

interpolates between greedy prioritisation and uniform random sampling by defining the probability of selecting an experience i as

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \quad \text{where} \quad p_i = |\delta_i| + \varepsilon \quad (2.17)$$

where p_i is the prioritisation value, ε ensure non-zero priority values, and α controls the degree of prioritisation. After each training iteration, the priority p_i of a revisited experience is updated with its new TD error value.

PER introduces bias by modifying the sampling distribution. To compensate for this, importance sampling (IS) weights are used to scale the gradient updates during training. The IS weight for a sample i is calculated as

$$\omega_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta \quad (2.18)$$

where N is the total number of experiences in the replay buffer and $\beta \in [0, 1]$ adjusts the strength of the correction. A value of $\beta = 0$ does not apply any correction (pure prioritisation), while $\beta = 1$ fully compensates for the bias, making training steps behave as if the experiences were sampled uniformly.

To gradually reduce bias over the course of training, a β annealing mechanism is applied. This decreases β from an initial value β_0 to a final value of 1, compensating for the bias introduced by PER at the end of the training.

2.4.4 DQN for area coverage with UAVs

As discussed in Section 2.4, recent surveys on the application of reinforcement learning to UAV networks [8] [2] denote the increase in research on deep learning methods for UAV swarm control and network planning. In particular, [8] identify the improvement of network coverage by UAVs as one of the main challenges and open research areas and highlight the importance of AI-based algorithms due to the non-human intervention nature of UAV operations.

In [2], the authors classify different applications of reinforcement learning in UAV networks by the class of DRL algorithms used, remarking the use of DQN algorithms in power allocation tasks. Of special interest for this work is research related to UAV placement as base stations in multi-hop scenarios. More specific those which uses high-dimensional representation of environment and CNN to process the grid input state such as the DQN application to Atari games from Google DeepMind [16].

An example of this is the work presented in [3], which proposes the use of Double-DQN (DDQN) to maximise the coverage of ground users (GUs) using UAVs as aerial base stations (ABSs). The proposed method employs a CNN to process a coverage bitmap encoding the position of the GUs and ABSs. This approach is centralised, predicting optimal actions for every UAV from a single DQN agent.

In contrast, [4] proposes a decentralised approach based on classical Q-Learning for emergency communications. In this work, the space is divided to discretise the continuous three-dimensional position of each UAV. The goal of the proposed algorithm is to maximise the number of users served by each drone, employing a global reward as the total number of users allocated by all base stations.

More recently, [30] introduced a DQN with CNN for flight planning and collision avoidance. In

this work, each UAV makes decisions based on local environment observation and provides a multi-resolution grid state to tackle the curse of dimensionality problem. The state grid is divided into a high-resolution central region for near agents and a lower-resolution external frame for more distant agents.

2.5 Communication technologies for UAV networks

A wide range of communication technologies can be employed to implement UAV-based networks. In [13], the authors review the application of UAVs to provide connectivity within three main types of networks: ad hoc networks, cellular networks, and satellite-based networks. They also divide UAVs roles in cellular networks into two categories: as user equipment (UE) or as aerial base stations (ABSs).

Flying ad hoc networks (FANETs) rely on different wireless communication standards. According to [31], common technologies include short- and medium-range standards such as Wi-Fi (IEEE 802.11), ZigBee, and Bluetooth; long-range low-power technologies such as LoRa and SigFox; and cellular technologies including 3G, 4G/LTE, and 5G.

2.5.1 Wireless ad-hoc networks

Wireless ad-hoc networks are especially relevant for UAV systems due to their decentralised nature and ability to operate without existing infrastructure. In such networks, each node operates both as host and router and employs dynamic routing protocols for multi-hop data transmission through a changing network topology.

Unlicensed wireless technologies such as Wi-Fi, ZigBee, and Bluetooth are widely used for small- to medium-scale UAV systems. The IEEE 802.11 Wi-Fi standard is the most widely used due to its balance between data rate and transmission range [32]. The typical Wi-Fi system has a transmission range of approximately 100 metres when operating in 2.4 GHz radio band. Alternatives to Wi-Fi are low-power, low-cost radios such as Bluetooth and ZigBee, which present a shorter range of 10 to 100 metres but have reduced power consumption.

Conventional Wi-Fi networks operate using two main modes: Access Point (AP) and Station (STA). In this setup, users (STA) connect to central AP, usually router, to access the wireless local area network (WLAN). In contrast, ad hoc networks use the IBSS (Independent Basic Service Set) mode, where all nodes communicate directly without a central coordinator. This configuration relies on ad hoc routing protocols to forward packets across the network through multiple hops. Two widely used protocols are:

- **AODV (Ad hoc On-Demand Distance Vector):** Reactive routing protocol that establishes routes only when required, reducing control overhead in highly dynamic topologies.
- **OLSR (Optimised Link State Routing):** Proactive routing protocol that maintains routes through periodic exchange of topology information between nodes.

An alternative to ad-hoc mode (IBSS) is Wi-Fi Mesh technology (IEEE 802.11s), which extends the IEEE 802.11 MAC standard to support self-configuring multi-hop topology. Wi-Fi Mesh depends on other Wi-Fi IEEE 802.11 standards to transport traffic and uses the Hybrid Wireless Mesh Protocol

(HWMP), based on AODV, as the basic routing protocol for the mesh network.

2.5.2 UAVs and cellular networks

Unmanned aerial vehicles (UAVs) can operate within cellular networks as user equipment (UE) or as aerial base stations (ABSs). In [5] authors identify several advantages of using UAVs as UE, including almost unlimited operational range and support for large numbers of users. The authors also highlight the use of device-to-device (D2D) communications enabled by LTE and 5G networks.

UAVs can also serve as aerial base stations. The 3GPP standardises the application of UAVs to assist 5G networks, designating them as UxNB nodes and defining three modes of operation: aerial base stations (ABSs), aerial relays (AR), or isolated ABS nodes [33]. Commercial LTE bases stations with a mas below 4 kg are already available, making airborne deployment a feasible possibility.

A significant benefit of cellular networks is their wide communication range. Traditional LTE base stations can cover areas of 2 to 4 km, with an extended coverage range of up to 30 km for rural environment when operating in low-frequency bands. 5G networks support a wider range of frequencies, from low-band with a large range for rural areas coverage to high-speed millimetre-wave bands optimised for urban scenarios with high density of users.

An example of an airborne LTE solution is the Patron-X10 from IWAVE [34]. This compact LTE base station (eNodeB) can provide coverage over a radius of 20 km when deployed at an altitude of 100 metres. It supports LTE in TDD mode and incorporates an Evolved Packet Core (EPC) multimedia dispatch server, designed for rapid deployment of private LTE networks in emergency scenarios. However, due to its weight of 5.5 kg and power consumption of 150 W, the equipment is designed for deployment with a tethered drone system, limiting its applicability for fully autonomous UAV systems.

2.6 UAV network simulation

The analysis of UAV-based networks presents several difficulties, including the modelling of complex path loss channels, the high mobility of UAVs, and the rapid changes in the network topology. To evaluate the performance of FANETs without conducting real-world experiments, simulation tools are employed. However, widely used UAV simulators, such as PX4 SITL simulation or X-Plane, do not support detailed network modelling, while network simulators like NS-3 and OMNeT++ do not natively support mobility models for the rapid dynamic of UAVs.

To address this, several simulation tools have been developed to implement, test, and evaluate UAV-based networks. In [31], the authors provide an extensive list of simulation platforms that support both UAV dynamics simulation and detailed network modelling. From this list, the following tools are outlined:

- **FlyNetSim:** UAV network simulator that combines the NS-3 network simulator with a lightweight publisher-subscriber communication scheme based on the ZeroMQ messaging library [6].
- **AVENS:** Simulation test bed that combines the X-Plane flight simulator with the OMNeT++ network simulator to provide realistic UAV and network modelling [35].

- **CUSCUS**: Simulator designed for network control systems based on FL-AIR and NS-3. It includes detailed environment simulation by generating obstacle maps from OpenStreetMap, which are used for UAV collision detection and realist channel modelling [36].
- **MARSNet**: Multi-agent UAV simulator created to support the development of the MARSNet communication protocol. It combines ROS/Gazebo simulator for robot systems with NS-3 through a modified version of FlyNetSim middleware [37].

Of particular interest for this work is the architecture presented in FlyNetSim [6]. This simulator integrates flight simulation with the NS-3 discrete event simulator by connecting both components through sockets and managing communication through the ZeroMQ library. In this approach, NS-3 acts as a simulation server, scheduling periodic polling events to read the socket. This mechanism enables exchange of mobility information between the simulators and facilitates the simulation of data packet traffic across the network.

Chapter 3

System design

As mentioned in 1.1, the main objective of the proposed system is to rapidly deploy a swarm of UAVs to restore wireless coverage in the mission area. UAVs, also known as drones, will act as relay nodes of an ad-hoc network and as base stations for users on the ground. The drones will be deployed and commanded from a ground control station (GCS), which will also serve as a recharging point when the drones are down on the battery. While UAV swarming and control is done by decentralised algorithms, the GCS will send commands to control the entire swarm behaviour or to request specific tasks to individual drones.

3.1 System architecture

There will be three types of agents involved in the system:

- **Drones or UAVs (Unmanned Aerial Vehicles):** These are the nodes that form the ad hoc network. They are deployed and controlled from a ground control station (GCS), to which they must return periodically for battery recharging. Each drone executes a distributed algorithm that determines its optimal position in order to maximise area coverage and ensure connectivity with the remainder of the swarm. They act as access points for users and connect with nearby drones to exchange information and transmit data packets.
- **Users or UEs (User Equipment):** These are the mobile terminals used by people who use the network for communication services, such as emergency personnel and civilians affected by the disaster.
- **Ground Control Station (GCS):** This serves both as a deployment point for the drones and as a station where the drones return to recharge their batteries. The GCS must maintain connectivity with the swarm as they send control commands to drones and act as gateways to the Internet or other emergency service networks.

The figure 3.1 illustrates the interaction between the various agents, including the access network links (green), the core network links (blue) and the path of a data packet (red).

Each agent has a network interface that enables communication with other nodes of the swarm network.

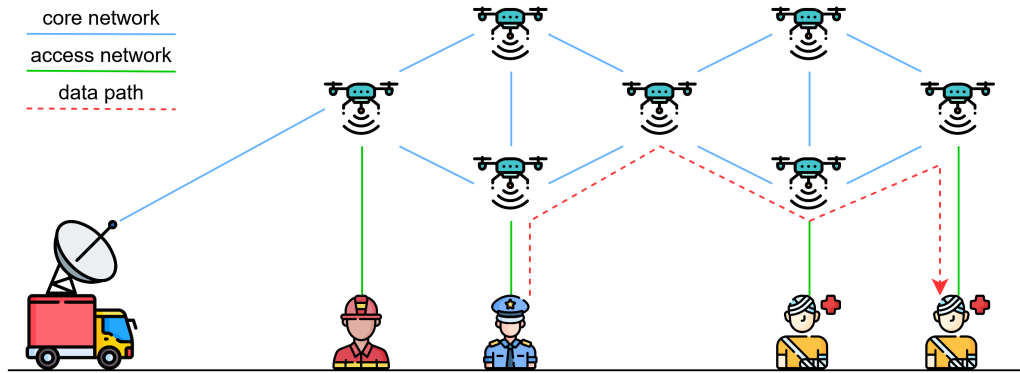


Figure 3.1: Diagram of system agents interaction.

For this purpose, a simple messaging protocol called SwarmLink was developed, as explained in Section 3.3. On top of each network interface, a SwarmLink interface is implemented to automate swarm protocol interaction. Users and drones automatically broadcast their positions, while GCS sends control commands and records position reports to visualise the swarm state. In addition, users can send messages to other users and receive a notification from the destination to confirm reception. Drones incorporate a swarming algorithm and a position controller, while GCS integrates the control application.

The GCS control application interacts with the swarming algorithms of each UAV to configure its behaviour (deploy, back to home, etc.) and configure mission parameters as operation area boundary or avoidance region coordinates. The UAV swarming algorithms interact between them by exchanging positioning information and interact with the position controller of each UAV to maintain the commanded position. Each user incorporates a messaging application whose purpose is to automate message reception and acknowledgement. This application interacts with the SwarmLink interface in order to communicate with other agents.

3.2 Technology selection

To create a realistic simulation and ensure the viability of the system, two critical design decisions must be made: the selection of the UAV model and the communications technologies used.

3.2.1 UAV model selection

The choice of drone model should be balanced between long flight range, compact size, and low operational cost of the system. The authors of the review [7] state that FANETs (Flying Ad-Hoc Networks) with multiple small UAVs have better advantages, including scalability, lower deployment cost, and better survivability compared to larger drones with longer flight times. The authors classify as small UAVs those with a flight weight below 25 kg.

To model the general characteristics of this class of UAVs, we review the specifications of three available quad-copter drones commonly used in enterprise applications. These drones are designed to carry a variety of payloads, including different types of cameras, sensors, and wireless transmitters, enabling

them to perform a wide range of missions. The selected UAVs and their specification are summarised in Table 3.1.

Table 3.1: Specifications of enterprise UAV models

Model	Weight	Max speed	Battery lifetime	Unit cost
DJI Mavic 3E	1 kg	15–21 m/s	30–45 min	\$3,300
DJI Matrice 30	4 kg	21 m/s	41 min	\$6,900
Freefly Astro	3–7 kg	–	38 min	\$22,500

For this simulator, the DJI Mavic 3E has been selected due to its lower cost, reduced weight, and maximum speed and battery lifetime similar to the other models. In the simulation, a target speed range of 10–20 m/s will be considered. Additionally, the 1 kg weight of the drone allows for simplification in the dynamic equations, as it can be treated as a unitary mass.

3.2.2 Communication standard selection

As reviewed in Section 2.5, there are a variety of wireless technologies to enable communication in UAV networks. Among them, cellular networks are especially interesting as they allow UAVs to integrate with existing mobile infrastructure. These technologies provide long communication ranges and do not require special hardware from users, just standard cellular devices, making them ideal for emergency scenarios.

However, as discussed in 2.5.2, commercial airborne nodes for LTE and 5G networks tend to be heavy and have a high power consumption. In addition, modern cellular networks have multilayered architectures, resulting in complex topologies that are difficult to implement in network simulation frameworks such as NS-3.

To simplify network design and validation in simulation environment, we propose using short- to medium-range communication standards. In Section 2.5.1, we reviewed Wi-Fi, ZigBee, and Bluetooth as radio access technologies for ad-hoc UAV networks. Among these, Wi-Fi is preferred due to its larger communication range and throughput compared to low-power technologies such as ZigBee or Bluetooth, while still having lower power consumption than LTE-based solutions.

Table 3.2: Comparison of selected Wi-Fi modules for UAVs

Feature	ESP32-WROOM-32	Skylab SKW77
Wi-Fi standards	IEEE 802.11 b/g/n	IEEE 802.11 b/g/n (2x2 MIMO)
Max data rate	150 Mbps	300 Mbps
Typical range	100 m	1000 m (with directional antenna)
Max TX power	20 dBm (100 mW)	28 dBm (630 mW)
RX sensitivity	-92 dBm	-165 dBm
Power consumption	160 mA @ 3.3V (528 mW)	740 mA @ 5V (3.7 W)
Price Range	\$3–6	\$30–45

There is a wide variety of Wi-Fi modules for UAVs, which offer a wide range of sizes, data rates, transmission powers, and energy requirements. Low-power modules such as the ESP32 are suitable

for small UAVs and short-range communication due to their compact size, low energy consumption, and reduced cost. These modules can be integrated directly into the drone's motherboard, simplifying hardware design. In contrast, high-power modules such as the Skylab SKW77 support long-range and high-throughput applications but require more power consumption, increased weight, and higher costs. Table 3.2 compares these two modules.

3.2.3 Communication range and number of UAVs

Wi-Fi technology comprises multiple standards such as IEEE 802.11b, 802.11g, and 802.11n, each using different coding and modulation schemes. The maximum communication range and data rate that can be achieved depends on the selected modulation and transmission power. To determine the most suitable Wi-Fi standard (IEEE 802.11b/g/n) and operational configuration, the following high-level system requirements are defined:

- Emergency communications do not require high data rates, only minimal performance is required to transmit short messages between network users.
- The transmission delay is not critical and may be in the order of 1 second.
- The system is designed to operate with a number of 10 to 100 UAVs. Fewer than 10 will be insufficient for effective operation of swarm algorithms, while more than 100 will increase system costs and may produce communication overhead.
- The swarm shall cover a mission area ranging from 1 km^2 to 100 km^2 . Smaller areas do not need UAV swarms to be covered, while larger areas may exceed system capacity.

As emergency communication does not require high data rates and delay is tolerable, the IEEE 802.11b standard operating in DSSS modulation at 1 Mbps is selected. This configuration will provide the maximum range, despite the lowest data rate.

Log-distance path loss model

To evaluate transmission power and range, the log-distance path loss model is applied:

$$L(d) = L_0 + 10n \log_{10} \frac{d}{d_0} \quad (3.1)$$

where L_0 is the reference path loss at distance d_0 , and n is the path loss exponent. For propagation in the free space, the loss exponent takes a value of $n = 2.0$, while indoor environments with high obstacles density present values of $n \approx 3.0$. A value of $n = 2.4$ is chosen to represent a medium-density urban environment.

The loss of the reference path is calculated for the 2.4 GHz channel 1 frequency of the Wi-Fi band of $f = 2412 \text{ MHz}$ and for a reference distance of 1 metre as

$$L_0 = 20 \log_{10} d_0 + 20 \log_{10} f + 32.44 \quad (3.2)$$

where d_0 is the distance in km and f the frequency in MHz, giving a reference loss of 40.09 dBm.

The maximum theoretical range for the devices proposed in Table 3.2 can be derived from its transmission power and receiver sensitivity. However, the real communication range is influenced by other factors such as SNR, modulation scheme, and coding rate, reducing the effective sensitivity.

Communication range tests

To estimate practical range, a sweep range test was performed in NS-3. The simulation test uses two nodes with IEEE 802.11b interface operating in ad hoc mode. One node acts as transmitter, sending dummy packets to the receiving node. The distance between both nodes increases incrementally in 5 m steps until reception fails. Transmit power levels of 20 dBm (100 mW) and 30 dBm (1 W) were evaluated, with a log-distance propagation model configured with $n = 2.4$ and a frequency of 2412 MHz (Wi-Fi channel 1). The results of the sweep test are as follows:

- For 20 dBm the maximum range was 381 m with an RSSI of -82 dBm.
- For 30 dBm the range increased to 995 m, also with an RSSI value of -82 dBm.

The default NS-3 Wi-Fi interface has a sensitivity of -101 dBm, indicating that the effective sensitivity is substantially lower in practice.

Minimum number of UAVs

To estimate the minimum number of UAVs required to ensure full area coverage, we assume a square-shaped mission area with UAVs deployed in a uniform grid. Each UAV occupies a grid node with separation from other UAVs corresponding to the maximum communication range.

Two representative scenarios are evaluated:

Small mission area $A = 1 \text{ km}^2$ with lowest transmit power (20 dBm):

- Communication range: $d = 381 \text{ m}$
- Area side length: $L = \sqrt{A} = 1 \text{ km}$
- Number of cells: $N = L/d = 2.62 \approx 3$
- Minimum number of drones of $M = (N - 1)^2 = 4$

Large mission area $A = 100 \text{ km}^2$ with maximum transmit power (30 dBm):

- Communication range: $d = 995 \text{ m}$
- Area side length: $L = \sqrt{A} = 10 \text{ km}$
- Number of cells: $N = L/d = 10.1 \approx 11$
- Minimum number of drones of $M = (N - 1)^2 = 100$

These calculations suggest that the system is theoretically viable and satisfies the proposed requirements. However, further validations are conducted in the simulation environment to assess the deploy-

ment of UAVs with proposed swarming algorithms and the reliability of data transmission through the ad hoc network.

3.3 SwarmLink protocol

SwarmLink is a simple protocol developed to support communication and coordination between agents (GCS, drones, and users) in the system. This protocol enables user message exchange messages and drone position reporting. Additional functionalities such as control commands, message acknowledgements, and the heartbeat mechanism have been considered and partially implemented, although not fully supported yet.

SwarmLink operates at the application layer, defining the content and format of the data packet. The current system implementation transmits packets over UDP sockets, using IPv4 as the network layer protocol along with AODV or OLSR for ad hoc routing. The network interface is based on the IEEE 802.11b Wi-Fi standard operating in AP mode with DSSS modulation at 1 Mbps data rate, as previously described in 3.2.2.

3.3.1 Position broadcast services

Drones periodically broadcast their positions using two reporting services: local and global. The local broadcast sends position updates at 10 Hz to immediate neighbours in a one-hop limited transmission, providing frequent data to facilitate collision avoidance. The global broadcast forward positions through the entire network at 1 Hz, offering full swarm status information while limiting network flooding. To avoid synchronised transmissions and reduce Wi-Fi collisions, a random jitter disperses the broadcasts uniformly over time. Each broadcast is scheduled at

$$t_{new} = t_{old} + T_{service}(1 + \omega) \quad (3.3)$$

where ω is a normal random variable with mean $\mu = 0$ and standard deviation $\sigma = 0.1$, producing $\pm 10\%$ timing variation.

The local broadcast service uses the limited IPv4 broadcast address 255.255.255.255 as the destination, to target all drones near the target. This address does not belong to the swarm network (10.0.0.0/16) and therefore it is not forwarded by ad hoc routing protocols, ensuring one-hop limited communication. The global broadcast service uses the IPv4 broadcast address of the swarm network (10.0.255.255), allowing reported position packets to be forwarded across all drones in the network using ad hoc routing protocols such as AODV or OLSR.

3.3.2 SwarmLink packet structure

SwarmLink packages are defined in binary format. Each swarm node implements a SwarmLink protocol interface which serialises outgoing packets to binary and deserialises incoming packets to structured data.

All SwarmLink packets begin with a header of 8 bytes (or 64 bits) with the following fields:

- **Packet Type** (1 byte): An 8-bit code to identify the packet type. Currently, there are 3 types

of packets defined: DATA (0x00), POSITION (0x01), and ACKNOWLEDGE (0xFF).

- **Agent ID** (1 byte): An 8-bit unsigned integer corresponding to the unique agent identifier. It allows defining up to 256 agents.
- **Packet ID** (2 byte): A 16-bit circular counter to recognise duplicated packets. Its value resets to 0 when it exceeds 65535.
- **Timestamp** (4 bytes): A 32-bit floating point with the current operating time of the system or absolute UNIX epoch timestamp (since January 1, 1970).

The general SwarmLink packet format, also corresponding to the structure of the data packet, is shown in Figure 3.2. Data packets can have a variable payload length that is used to transport message data in binary format.

Packet Type (1 byte)	Agent ID (1 byte)	Packet Identifier (2 bytes)
Timestamp (4 bytes)		
Payload (N bytes)		

Figure 3.2: SwarmLink header format

The position data packets, designed by packet code 0x01, contain the 3-dimensional relative position of the drone to the GCS in meters, encoded as three 32-bit floating-point variables, giving a total payload size of 12 bytes.

Packet Type (0x01)	Agent ID (1 byte)	Packet Identifier (2 bytes)
Timestamp (4 bytes)		
Position X (4 bytes)		
Position Y (4 bytes)		
Position Z (4 bytes)		

Figure 3.3: SwarmLink position packet format

This representation provides enough precision for swarm operation. A 32-bit floating-point provides 24-bit of precision, about 7 decimal digits of precision. This provides a resolution of 6 *cm* for a 10^6 *m*, which is much more than expected operation range. Considering that typical operation range is between 1 *km* to 10 *km* and positions are represented as relative to control station (GCS) or home position, the expected resolution is between 0.06 *mm* and 0.6 *mm*.

Chapter 4

Swarming algorithms

This project is focused on the emergency communication scenario described in [9]: reestablishment and overhaul of mobile services. For this, a group of quadcopters, acting as base stations, will be deployed under the control of a distributed swarming algorithm. The purpose of this algorithm is to guide drones to their optimal position to maximise area coverage, while avoiding collision with obstacles and promoting uncovered area exploration.

Two swarming algorithms are studied and implemented for this purpose: the Extended Virtual Spring Mesh (EVSM) from [1] and an original reinforcement learning approach based on the popular DQN algorithm, first introduced by DeepMind in [25].

4.1 Modified EVSM algorithm

The EVSM algorithm introduced in [1] was shown to be unstable in our simulation environment. The collision avoidance mechanism of the original formulation proved to be ineffective, unable to navigate obstacles. In addition, the edge drone tests were revised and a simpler and more efficient implementation is provided. The main improvements of the modified EVSM algorithm are as follows:

- Implementation of a collision avoidance mechanism with a potential force inspired by gravity and electromagnetic fields.
- Reformulation of the edge robot test with a simpler algorithm based on the comparison of relative sweep angles.
- Inclusion of increasing natural length for the expansion mechanism during deployment.
- Separation of the damping force to simplify tuning.
- Performance optimisation of the EVSM algorithm by limiting the spring calculation to near-neighbours and running at a lower update frequency.

4.1.1 Potential collision avoidance force

The collision avoidance mechanism generates a repulsion force when a drone gets close to an obstacle. This force is implemented as a normalised potential field that is activated when a minimum distance to obstacles is violated. The formulation is as follows:

$$\mathbf{F}_{\text{coll}} = \sum_i k_c \frac{d_{\min}^2}{d_{\text{obs},i}^2} (-\hat{\mathbf{u}}_i) \quad (4.1)$$

where $d_{\min}$ is the minimum distance to obstacles, $d_{\text{obs},i}$ is the distance to the i -th obstacle, and $\hat{\mathbf{u}}_i$ is the direction from the drone to the i -th obstacle.

The current implementation with a minimum distance to obstacles of 10 metres proved to be highly effective in simulation environment. The collision avoidance force increases rapidly as the robot approaches the obstacle, until reaching an infinite force at zero distance. To limit the force value and avoid the direction from becoming undefined by infinite magnitude, the force constant is clipped to a maximum value set by the maximum allowed acceleration. The current setup uses a maximum acceleration 10m/s^2 (or unitary force) and only calculates the collision avoidance force when the robot is closer to obstacles than the minimum distance (10 metres), reducing the required calculations when far from obstacles and improving the performance of the algorithm, as proposed in [1].

4.1.2 Modified edge robot test

As explained in Section 2.3.3, the exploration mechanism of the EVSM algorithm applies an attractive force to far obstacles on edge drones [1]. In the original formulation, a robot i is considered an edge robot if its sweep angle θ_i is greater than 90° and a polygon projected from the sweep angle bisector does not contain any other robot inside. However, this approach requires using the ray casting algorithm to determine if a point is inside or outside the polygon, increasing computational complexity. To reduce this complexity and improve the performance of the algorithm, we propose a simplified edge robot test based solely on the computation of the sweep angle.

The sweep angle, as defined in [1], is the largest angular range in which no neighbour is visible from robot i . To calculate this angle, the relative positions r_{ij} of nearby drones j are used to determine their corresponding angular positions θ_{ij} , where

$$\theta_{ij} = \text{atan2}(r_{y,ij}, r_{x,ij}). \quad (4.2)$$

These angles are then sorted in ascending order and the angular differences between successive angles are calculated. The sweep angle θ_i is defined as the largest of these angular differences.

The sweep angle returned by Algorithm 1 is a tuple of angular bounds indicating the start and end of the largest angular difference. The value of the sweep angle is obtained by computing the normalised angular difference in $[0, 2\pi)$ range:

$$\theta_{\text{sweep}} = (\theta_{\text{end}} - \theta_{\text{start}}) \bmod 2\pi. \quad (4.3)$$

Algorithm 1: Calculate Sweep Angle

Input: Robot position $\mathbf{p} \in \mathbb{R}^2$, Neighbour positions $\mathbf{M} \in \mathbb{R}^{N \times 2}$
Output: Sweep angle range $(\theta_{\text{start}}, \theta_{\text{end}})$
if $N = 0$ **then**
 return $(0, 2\pi)$;
end
foreach neighbour $\mathbf{m}_j$ **in** $\mathbf{M}$ **do**
 $\mathbf{r}_j \leftarrow \mathbf{m}_j - \mathbf{p}$;
 $\theta_j \leftarrow \text{atan2}(r_{j,y}, r_{j,x})$;
end
Sort angles θ_j in ascending order;
for $j = 1$ **to** N **do**
 Compute angular differences:
 $\Delta_j \leftarrow (\theta_j - \theta_{j-1}) \bmod 2\pi$;
end
 $i \leftarrow \arg \max_j \Delta_j$;
 $\theta_{\text{start}} \leftarrow \theta_{i-1}$;
 $\theta_{\text{end}} \leftarrow \theta_i$;
return $(\theta_{\text{start}}, \theta_{\text{end}})$;

In the proposed test, a drone is identified as an edge drone if this sweep angle exceeds 90° ($\theta > \pi/2$). For the special case where the drone does not have any neighbours, the sweep angle is considered to be 2π , as indicated by identical start and end angles.

The exploration force described in Equation 2.2 uses the calculated sweep angle to determine which obstacles are within the field of view of an edge drone. From each obstacle i , the direction vector $\hat{\mathbf{u}}_i$ from the drone to the obstacle is used to calculate its angular position:

$$\beta_i = \text{atan2}(u_{i,y}, u_{i,x}) \quad (4.4)$$

The relative difference between the angular position of the obstacle β_i and the start of the drone sweep angle θ_{start} is calculated as

$$\Delta\beta_i = (\beta_i - \theta_0) \bmod 2\pi \quad (4.5)$$

If this difference $\Delta\beta_i$ falls within the range of $[0, \theta_{\text{sweep}}]$, where θ_{sweep} is the angular width of the sweep angle, the obstacle i is considered to be within the field of view of the edge drone.

4.1.3 Improved acute angle test

As discussed in Section 2.3.2, the acute angle test proposed in [15] provides a planar and connected graph for any spatial distribution of drones. The algorithm runs with a time complexity of $O(N^2)$, where N is the number of neighbour drones. This quadratic complexity is explained by the requirement that each drone must perform the acute angle test against all neighbours i , and each test involves

validating whether any other neighbour $j \neq i$ is within the circle defined by the diameter of the segment between the drone and the neighbour i under test.

In practice, the acute angle test runs about 10 times faster than the worst-case quadratic complexity, as explained in [15]. This performance improvement is due to early termination within the inner loop when a neighbour j falls within the circle, violating the acute angle condition and thereby reducing the number of required computations.

Algorithm 2: Acute Angle Test

Input: Drone position $\mathbf{p} \in \mathbb{R}^2$, Neighbour positions $\mathbf{M} \in \mathbb{R}^{N \times 2}$
Output: Springs boolean vector $\mathbf{L} \in \{\text{True}, \text{False}\}^N$
Initialise springs vector: $\mathbf{L} \leftarrow$ array of N False values;
for $i \leftarrow 1$ **to** N **do**
 Calculate circle for drone and i -th neighbour:
 $\mathbf{c} \leftarrow (\mathbf{p} + \mathbf{m}_i)/2$;
 $r \leftarrow \|\mathbf{p} - \mathbf{m}_i\|/2$;
 for $j = 1$ **to** N **do**
 if $j = i$ **then**
 | skip this step
 end
 Calculate distance between circle centre and j -th neighbour:
 $d \leftarrow \|\mathbf{m}_j - \mathbf{c}\|$;
 if $d \leq r$ **then**
 | break;
 end
 end
 if $d > r$ **then**
 | Create spring to i -th neighbour:
 | $\mathbf{L}[i] \leftarrow \text{True}$;
 end
end
return $\mathbf{L}$;

The implementation we propose in Algorithm 2 is consistent with the formulation and results described in [1], although no explicit pseudocode was provided there. The acute angle test is the most computationally intensive component of the EVSM algorithm, as its complexity increases quadratically with the number of simulated agents. To mitigate this, we propose an optimisation by reducing the update rate of the spring connectivity matrix.

In the current simulation, agent dynamics and network interfaces are updated at 100 Hz (every 10 ms), while the control algorithm (EVSM or SDQN) operates at 10 Hz (every 100 ms). By reducing the update rate of spring formation to 1 Hz (every 1 s), the number of acute angle test evaluations is significantly reduced, improving overall simulation performance. This optimisation proved to have no negative impact on the simulated environment, as the spring mesh does not require as frequent updates as the calculation of repulsive or attractive forces to nearby neighbours or obstacles.

4.1.4 Modified algorithm description

The Algorithm 3 describes the detailed implementation of the modified EVSM algorithm, which was developed through several design iterations and performance optimisations. These include updating the control force at intervals of 100 ms, and recalculating springs mask and sweep at a lower interval of 1 second.

The expansive natural length is a mechanism introduced to accelerate drone deployment during the initial phase of mission. This approach resembles the expansion force mentioned in [1], although no specific implementation was provided there. In our implementation, the natural length l_0 is set at an initial value of l_{min} and increases at a constant rate l_{rate} until it reaches a final value l_{max} . This relation is formulated as follows:

$$l_0(t) = \max(l_{min} + l_{rate}(t - t_0), l_{max}) \quad (4.6)$$

If the distance from the drone to the nearest obstacle d_{obs} is less than the critical distance d_{min} (defined in the current implementation as 10 metres), the collision avoidance force is applied and the resulting control force is returned, avoiding the control force and calculation of the exploration force. This approach has proven to be more effective for the avoidance of obstacles, as it prioritises obstacles repulsion over control or exploration when critical separations occur d_{min} .

An additional force-limiting mechanism has been implemented to ensure realism of drone dynamics and avoid unrealistic extremely high acceleration values. If the magnitude of the computed control force $\|\mathbf{u}\|$, exceeds the maximum force value u_{max} , it is clipped as

$$\mathbf{u} \leftarrow u_{max} \cdot \frac{\mathbf{u}}{\|\mathbf{u}\|} \quad (4.7)$$

It is import to note that all forces are expressed as acceleration by assuming a drone unitary mass of 1 kg. This simplification is common in control theory and robotics, and is often referred to as a normalised control model. This approach simplifies controller design and analysis by removing the need to consider the mass in control equations.

4.1.5 Position and altitude control

The spring mesh and damping forces in the EVSM algorithm correspond to a mass-spring-damper system. This can be interpreted as a proportional-derivative (PD) controller, where the proportional error value is defined as the difference of the spring length from its natural length: $e_d = l_0 - d$, where l_0 (natural length) represents the desired distance and d the current distance. The derivative error is given by $\dot{e} = 0 - v$, where the desired velocity is zero to ensure a convergence of the system to a static mesh formation. In this interpretation, the spring constant k_s corresponds to the proportional gain k_p , while the damping coefficient k_d functions as the derivative gain.

The EVSM algorithm provides two-dimensional control forces to regulate horizontal position. Vertical control of drone is managed through a height hold mechanism. Height h is defined as the vertical distance of the UAV above the ground surface, while altitude z refers to the distance above mean sea

Algorithm 3: Modified EVSM Control Algorithm

Input: Current time $t \in \mathbb{R}$, Drone position $\mathbf{p} \in \mathbb{R}^2$, Drone velocity $\mathbf{v} \in \mathbb{R}^2$, Neighbour positions $\mathbf{M} \in \mathbb{R}^{N \times 2}$

Output: Control force $\mathbf{u} \in \mathbb{R}^2$

Initialise variables:

- Control force: $\mathbf{u} \leftarrow \mathbf{0}$;
- Springs mask: $\mathbf{L} \leftarrow$ array of N False values;
- Sweep angle bounds: $(\theta_{\text{start}}, \theta_{\text{end}}) \leftarrow (0, 0)$;
- Last update times: $t_{\text{control}} \leftarrow 0, t_{\text{springs}} \leftarrow 0$;

Update natural length: $l_0 \leftarrow \max(l_{\min} + l_{\text{rate}}(t - t_0), l_{\max})$;

Calculate elapsed time from last control update: $\Delta t_{\text{control}} \leftarrow t - t_{\text{control}}$;

if $\Delta t_{\text{control}} < 0.1$ *seconds* **then**

- Return previous control force value:
- return** $\mathbf{u}$;

end

Reset control force value: $\mathbf{u} \leftarrow \mathbf{0}, t_{\text{control}} \leftarrow t$;

Calculate elapsed time from last springs update: $\Delta t_{\text{springs}} \leftarrow t - t_{\text{springs}}$;

if $\Delta t_{\text{springs}} \geq 1.0$ *seconds* **then**

- Update springs mask and sweep angle:
- $\mathbf{L} \leftarrow \text{AcuteAngleTest}(\mathbf{p}, \mathbf{M})$ (Algorithm 2);
- $(\theta_{\text{start}}, \theta_{\text{end}}) \leftarrow \text{CalculateSweepAngle}(\mathbf{p}, \mathbf{M})$ (Algorithm 1);
- $t_{\text{springs}} \leftarrow t$;

end

Apply damping force: $\mathbf{u} \leftarrow \mathbf{u} - k_d \cdot \mathbf{v}$;

Find nearest obstacle: $d_{\min} \leftarrow \min_k \|\mathbf{o}_k - \mathbf{p}\|$ for all $\mathbf{o}_k \in \mathbf{O}$;

if $d_{\min} < d_{\text{obs}}$ **then**

- Apply collision avoidance force: $\mathbf{u} \leftarrow \mathbf{u} + \mathbf{F}_{\text{coll}}$ (Equation 4.1);
- return** $\mathbf{u}$;

end

foreach *neighbour* $\mathbf{m}_j$ *in* $\mathbf{M}$ **do**

- if** $\mathbf{L}[j] = \text{True}$ **then**
- Compute spring displacement: $\mathbf{r}_j \leftarrow \mathbf{m}_j - \mathbf{p}$;
 - Apply control force: $\mathbf{u} \leftarrow \mathbf{u} + k_s \cdot (\|\mathbf{r}_j\| - l_0) \cdot \frac{\mathbf{r}_j}{\|\mathbf{r}_j\|}$ (Equation 2.1);
- end**

end

if $\theta_{\text{sweep}} = (\theta_{\text{end}} - \theta_{\text{start}}) \bmod 2\pi > \pi/2$ **then**

- Apply exploration force: $\mathbf{u} \leftarrow \mathbf{u} + \mathbf{F}_{\text{expl}}$ (Equation 2.2);

end

return $\mathbf{u}$;

level (MSL), calculated as the sum of the local ground elevation H and the height of the UAV h above it.

To implement height hold, an altitude PD controller is employed, defined as

$$u_z = k_p \cdot (z_t - z) + k_d \cdot \dot{z} \quad (4.8)$$

where u_z is the control input along the z-axis, k_p is the proportional gain, z_t is the target altitude, z is the current altitude, k_d is the derivative gain and $\dot{z}$ is the velocity component of the z-axis. To maintain a constant height above the terrain, the target altitude is updated in each iteration as $z_t = h_t + H$, where h_t is the desired height above the ground level.

The selection of k_p and k_d gains is based on standard control tuning principles, as described in [38]. The proportional gain k_p is adjusted to provide the maximum control input value u_{max} at the maximum position error e_{max} as

$$k_p = \frac{u_{max}}{e_{max}} \quad (4.9)$$

The derivative gain k_d is set to achieve critical damping:

$$k_d = 2\sqrt{k_p} \quad (4.10)$$

This approach is applied to configure the altitude controller and the EVSM parameters, where the spring constant k_s is the proportional gain k_p , and the damping coefficient k_d is the derivative gain for the PD controller.

4.1.6 Performance validation

To evaluate the performance of the modified EVSM algorithm, a simulation was performed within a $200\text{ m} \times 200\text{ m}$ test environment. This environment presents a flat terrain with zero elevation and includes three obstacles of different shapes. A total of 36 drones are deployed from a grid formation 6×6 , with a spacing of 5 m and an origin in the lower left corner. The initial natural length is set to 5 metres and gradually increases to a final value of 100 metres with an expansion rate of 1 metre per second.

The maximum acceleration allowed is set to 10 ms^{-2} , and the maximum position error is defined as 100 metres. Based on these values, the proportional gain is calculated as $k_p = 0.1$, and the derivative gain for the critical damping as $k_d = 2\sqrt{k_p} \approx 0.6$. These gain values are close to the spring constant and the damping coefficient proposed in [1].

Figure 4.1 illustrates the evolution of the test simulation in the initial stage, after 30 seconds, and after 120 seconds, when the entire mission area is covered. The results demonstrate successful deployment of the swarm, the formation of a fully connected planar mesh with approximately the desired inter-drone separation, and effective obstacle avoidance. The background is coloured according to the strength of

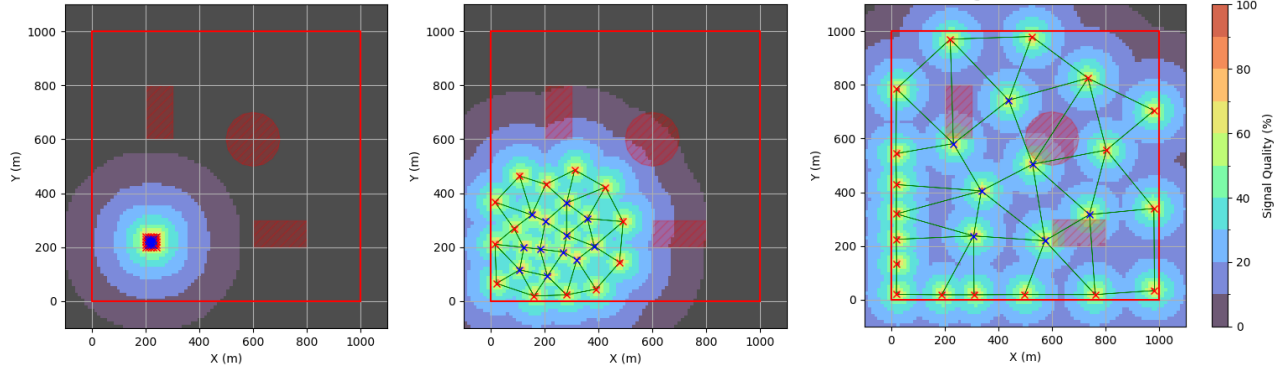


Figure 4.1: EVSM test at simulation time 0 s (left), 30 s (middle), and 120 s (right).

the received signal at ground level, with RSSI values scaled from -80 dBm to -30 dBm, corresponding to a signal quality from 0% to 100%.

4.2 Swarm Deep Q-Network

The Extended Virtual Spring Mesh (EVSM) algorithm, described in 4.1, can provide coverage of the entire mission area, while avoiding obstacles and ensuring a fully connected network. However, this algorithm may not be the most efficient for the deployment of UAV networks in certain scenarios. EVSM covers the whole area using a uniform mesh of drones, regardless of the distribution of users. As a result, it can require an unnecessarily large number of drones in scenarios where user locations are sparse. In addition, environments with many obstacles can extend the EVSM time to achieve full deployment.

To address these limitations, we propose the Swarm Deep Q-Network (SDQN) algorithm, which is based on multi-agent reinforcement learning techniques (MARL) and built over the Deep Q-Network (DQN) framework. SDQN can learn optimal UAV deployment strategies to adapt to complex environments, providing efficient user coverage when avoiding obstacles and maintaining full network connectivity. This deep reinforcement learning (DRL) approach can achieve a faster and more efficient deployment of UAV networks than EVSM, reducing the number of required drones and reducing the system cost.

The design of reinforcement learning (RL) systems is driven by the selection of the rewarding scheme, which influences the policy learnt by the agent. This can be a difficult task, as it is not clear what information must be provided to the agent or how to define rewards in order to achieve desired behaviour. If the state representation lacks information, the agent may be unable to learn properly. If the reward mechanism is poorly designed, the agent will find ways to maximise its rewards without obtaining the desired behaviour. Designing RL systems involves trial and error approaches and is sometimes considered more of an art than a science.

Taking these challenges into account, several design iterations were conducted. The following sections describe the different options explored for each component of the system and the ones chosen with the best performance.

4.2.1 SDQN agent

In the proposed system, each drone is equipped with an SDQN agent, which determines the target position based on learnt policy. The rewarding scheme is designed to maximise the number of users covered by the UAV swarm, while avoiding collisions with obstacles or nearby drones and maintaining a fully connected network.

The SDQN system consists of three main components: the frame generator, the DQN agent, and the action selector. The frame generator provides a pixel-based representation of the local environment, encoding the position of nearby users, obstacles, and neighbour drones. This image-based state representation is used as an input of the Q-network, which estimates the expected reward for taking each possible action. The action selector chooses the next action based on the current operating mode, such as training or deployment. During training, an epsilon-greedy policy is applied to balance exploration and exploitation. In contrast, a full exploitation policy is preferred during deployment. The selected action corresponds to a new target position, which is passed to the drone position controller.

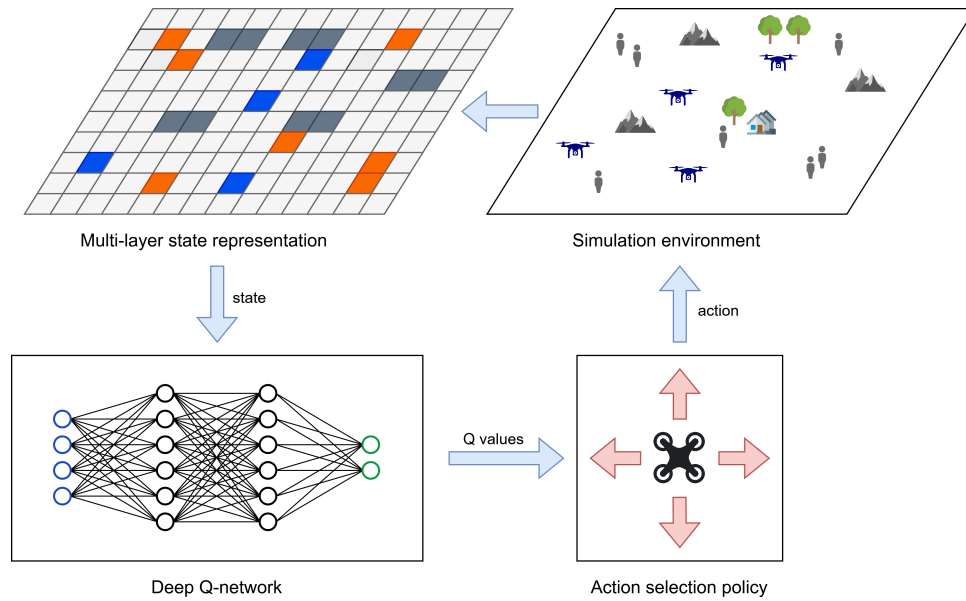


Figure 4.2: Swarm DQN agent components interaction

The SDQN agent is built on a double deep Q-Network (DDQN) architecture [28], which uses two separate networks to stabilise the learning process. In addition, a Prioritised Experience Replay (PER) buffer [29] is used to improve training efficiency. These components are discussed in detail in Sections 2.4.3 and 2.4.3, respectively.

The SDQN neural network is based on the original DeepMind Atari DQN [16]. It uses convolutional layers (CNN) to extract patterns from high-dimensional data and fully connected layers to estimate Q values. The network input is a 64x64x3 image, where each channel represents obstacles, drones, and users, respectively. The output layer has 5 or 7 units, depending on the selected action space, which are basic movement actions (up, down, left, right, or no movement) or an extended subset including zoom-in and zoom-out actions.

Table 4.1: CNN model of the DQN agent, based on [16].

Layer	Type	Details	Output Shape
Input	Image	$64 \times 64 \times 3$ (uint8 format)	$64 \times 64 \times 3$
Rescale	Normalization	Convert 0-255 (uint8) to 0.0-1.0 (float32)	$64 \times 64 \times 3$
Conv1	Conv2D	32 filters, 8×8 kernel, stride 4, ReLU	$15 \times 15 \times 32$
Conv2	Conv2D	64 filters, 4×4 kernel, stride 2, ReLU	$6 \times 6 \times 64$
Conv3	Conv2D	64 filters, 3×3 kernel, stride 1, ReLU	$4 \times 4 \times 64$
Flatten	Flatten	Convert to 1D vector	1024
FC1	Dense	512 units, ReLU	512
Output	Dense	5 or 7 units (actions), Linear	5 or 7

4.2.2 State representation

The state is encoded as a 3-channel image of size 64×64 pixels representing each drone's local environment. Each channel encodes a different feature: obstacles, neighbour drones, and users. The image is centred on the drone, so the centre pixel corresponds to the drone's own position. The surrounding pixels reflect the relative positions of other elements in the environment.

Each pixel uses an 8-bit value (0-255) to represent the intensity. To construct the frame, the absolute positions of the users, neighbours, and obstacles are converted into relative coordinates. The frame is a square image of $N \times N$ pixels with a side length of $L = 2R$, where R is the maximum vision radius. The spatial resolution of each pixel is $d = L/N$ in metres.

The relative position $\mathbf{p} = (p_x, p_y)$ for a pixel with index (i, j) is calculated as

$$\mathbf{p} = (p_x, p_y) = d \cdot (i, j) - R \quad (4.11)$$

The inverse conversion, from relative coordinate $\mathbf{p}$ to pixel indices (i, j) requires applying the floor operation for discretization:

$$(i, j) = \left\lfloor \frac{\mathbf{p} + R}{d} \right\rfloor \quad (4.12)$$

To obtain the relative position of an object in the frame, the current UAV's position $\mathbf{p}_{uav}$ is subtracted from the object's absolute position $\mathbf{p}_{obj}$:

$$\mathbf{p}_{rel} = \mathbf{p}_{obj} - \mathbf{p}_{uav} \quad (4.13)$$

The swarm drones have access to the absolute position of other drones, users, and obstacles. This information may be obtained through the simulated network or directly provided in idealised scenarios for testing purposes. The goal of state representation is to process local information in a compact format suitable for input to a neural network. Depending on the frame generation technique, high-resolution details of near objects or information about distant features of the environment may be lost.

Fixed-frame generator

The initial design used a fixed-size frame generator with a vision radius of $R = 500\text{ m}$, resulting in a $1\text{ km} \times 1\text{ km}$ frame. covering an area of $A = 1\text{ km}^2$. The channels were defined as follows:

- **Channel 1:** Binary collision risk map (1 if an obstacle or neighbour drone is present and 0 if not).
- **Channel 2:** Signal quality received from neighbour drones, scaled from 0 (−80 dBm) to 255 (−30 dBm).
- **Channel 3:** Signal quality received from users, using the same scale.

Figure 4.3 shows the fixed-frame state representation. The left image displays the entire mission area of $2\text{ km} \times 2\text{ km}$, where the red cross represents drone 0, blue crosses indicate the other drones, magenta points mark users on the ground, red boxes denote obstacles, and the heatmap visualises the signal quality of each drone, ranging from 0% to 100%.

The next three frames represent: (i) the collision risk map, (ii) the drone signal quality heatmap, and (iii) the user signal quality heatmap, each covering a $1\text{ km} \times 1\text{ km}$ area. In these local frames, the central coloured pixel corresponds to the position of drone 0, providing a vision range of 500 metres in all directions.

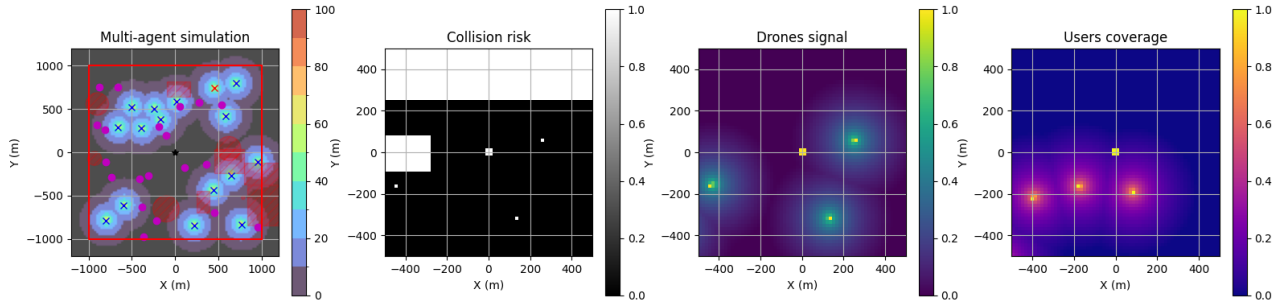


Figure 4.3: Fixed-frame generator outputs from multi-agent simulator

The signal quality is an indicator of the maximum received signal strength (RSSI) from multiple transmitters. Instead of summing all received signals, which is used to calculate interference level, the maximum RSSI is preferred to emulate the real behaviour of Wi-Fi receivers, which typically lock into the strongest signal. A linear scale is applied to map RSSI values from −80 dBm to −30 dBm in a range of 0% to 100%. The lower value of −80 dBm corresponds to the effective receiver sensitivity estimated in 3.2.3, while the upper level of −30 dBm represents the typical received power at close range and with minimal path loss.

The use of neighbour drones signal helps the agent learn efficient coverage strategies by avoiding overlap and maintaining connectivity with the swarm network. This encourages drones to position in poorly covered areas, while avoiding zones with no signal that could cause disconnection from the network. On the other hand, the user signal map helps drones to learn which users to cover. Since wireless communication is reciprocal, a strong signal received from a user implies that the drone is providing effective coverage to them. In addition, user signals help drones locate users outside the visible frame when position information is available.

Dynamic zoom mechanism

Frame size and vision radius are critical factors for SDQN performance. In large mission scenarios (areas with side lengths of 10 *km*), a fixed frame may be too local to provide enough environment information. A small frame limits situational awareness, while a large frame reduces pixel spatial resolution, missing small details needed for collision avoidance. To address this problem, we implemented a dynamic zoom mechanism by adding two new actions (zoom-in and zoom-out) to the action space. These actions adjust the vision radius by $\pm 20\%$ within a range of 100 *m* to 10 *km*, allowing the drone to adapt the field of view to different scenarios.

Multi-resolution frames

The dynamic zoom mechanism did not perform as expected during training, as drones tend to learn a large vision radius to gain vision of the entrance mission area. This increases the collision rate with near drones as the small details needed for effective collision avoidance are lost. To mitigate this, we explored multi-resolution state representations:

- **Hierarchical multi-resolution representation:** Based on [30], the frame is split in two zones: a high-resolution detailed view of near environment and a mean-field representation of distant features. This maintains detail for the local area while including information of distant objects.
- **Log-polar representation:** Inspired by human vision [39], this approach uses log-polar coordinates to achieve high resolution for near details while still capturing distant features. The frame is divided into radial ρ and angular θ components, similar to classical polar coordinates. However, the radial component is logarithmic $\rho = \ln \sqrt{p_x^2 + p_y^2}$, which produces a gradual compression of spatial information as distance increases.

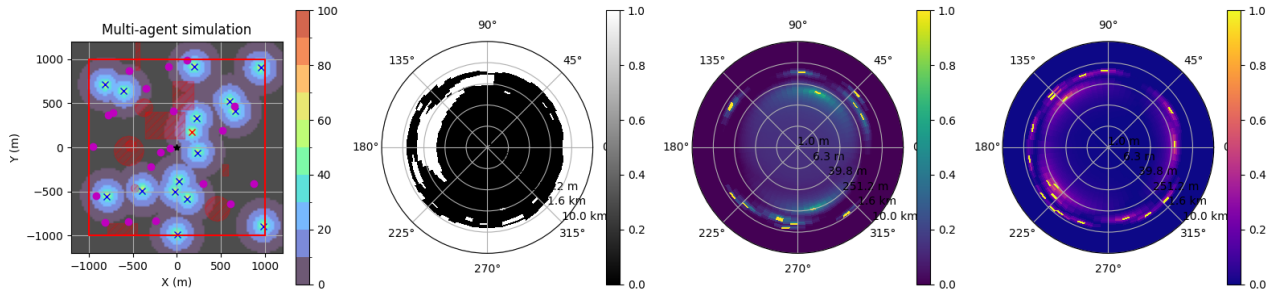


Figure 4.4: Log-polar frame generator outputs

As shown in Figure 4.4, the log-polar representation compresses distant features while preserving high detail for near environment. In this example, a visible radius is used from 1 *m* to 10 *km*. The first frame (black and white) shows the collision risk map. Areas farther than 1 *km* appear in white because they fall outside the mission boundary, indicating that the entire mission area is included within the frame. In addition, distances to near obstacles, such as the one on the left of the drone, are captured in high detail because of the high resolution of the central pixels.

This provides a more efficient input representation for the DQN compared to dynamic zoom, as it

maintains the fine-grained spatial detail required for obstacle avoidance while also capturing information about distant users and neighbour drones. Unlike hierarchical methods, which divide the input into discrete spatial zones, the log-polar encoding enables a smooth spatial transition, preserving context across scales in a continuous manner.

4.2.3 Reward scheme

The reward scheme is the method used to condition the learnt policy towards the desired behaviour. It consists of three types of reward: a collision risk penalty, a user coverage reward, and a full connectivity reward.

Collision risk penalty

The obstacle collision penalty encourages agents to avoid collisions with obstacles, mission boundaries, and neighbouring drones. A minimum separation distance of 10 metres is defined. If an agent is closer than this threshold to another drone or obstacle, the reward is set to -1 . If the distance to the nearest object is zero or less, a collision is considered to have occurred, the reward is set to -10 , and the episode is terminated (done = true). If the drone distance to the nearest object is equal to or less than 0, the collision is defined and the reward is set to -10 and done to true. This penalty mechanism was proven to work more effectively when combined with other rewards than simply summing the negative penalty because it encourages drones to prioritise collision avoidance to ensure survival.

User coverage reward

Three different mechanisms were evaluated to promote user coverage:

Global reward: The first approach involved providing a global reward based on the total number of users covered. The value between 0 and 1 represents the ratio of covered users to the total number of users. A user is considered covered if it receives a signal quality greater than zero from any drone. If the RSSI is below -80 dBm, the signal quality is zero, and the user is considered not covered. However, this approach was unsuccessful, as drones could not effectively learn a coordinated strategy from a global reward in multi-agent reinforcement learning (MARL) scenarios, as stated by [40].

Individual coverage reward: The second approach used individual rewards proportional to the number of users covered by each drone. To encourage drones to cover different users, each user was assigned to the drone that provides the strongest signal quality, preventing drones from gaining reward for users already covered by others. However, this method proved unstable, as drones try to 'steal' coverage from neighbours or focus on covering a single user with maximum signal strength to maintain their rewards, failing to provide effective coverage when the number of users was smaller than the number of drones.

Difference reward: The final mechanism employs difference rewards, inspired by [40] and originally introduced in [41]. This method assigns individual rewards that encourage the achievement of global objectives. The individual user coverage reward for the drone i is calculated as

$$D_i(s) = R(s, u) - R(s, u_{-i}) \quad (4.14)$$

where $R(s, u)$ is the global reward representing the total ratio of users covered with all agents u , and $R(s, u_{-i})$ is the global reward calculated after the drone i was removed from the simulation. This reward evaluates the contribution of the drone i to the global objective, taking a value of zero if the drone does not contribute to global user coverage. By maximising D_i , each agent improves both its own reward and the global reward, successfully learning individual policies that align desired behaviour.

Connectivity reward

To maintain a fully connected swarm network, two reward strategies were tested:

Direct connectivity penalty: In the first approach, drones receive a penalty of -1 if they do not detect any signal from other drones ($\text{RSSI} < -80$ dBm). This encourages drones to stay close to at least one neighbour. However, this method only ensures direct connections, as drones could form small isolated clusters that are not connected to the rest of the swarm. This strategy was proved ineffective as the network usually becomes fragmented.

Global connectivity reward: To ensure global connectivity, a difference reward based on the largest connected cluster is used. First, the connection matrix is calculated based on which drones were within the communication range of each other. Then, the Depth-First Search (DFS) algorithm [42] is applied to find all connected components.

The largest group of connected drones is considered the main swarm network. The global reward is the number of drones in the largest cluster divided by the total number of drones. The individual reward for each drone is calculated using the difference reward as

$$D_i(s) = R(s, u) - R(s, u_{-i}) \quad (4.15)$$

where $R(s, u)$ is the global reward for all drones, and $R(s, u_{-i})$ is the reward if the drone i is removed. This encourages each drone to maintain the overall network structure by contributing to the main connected group.

Total reward function

The total reward function combines global objectives (user coverage and connectivity) with collision avoidance. The final reward R_i for each drone i at each step is calculated as

$$R_i = \begin{cases} D_i^{\text{cover}} + D_i^{\text{conn}}, & \text{if } d_{i, \text{obs}} > d_{\min} \\ -1, & \text{if } d_{i, \text{obs}} \leq d_{\min} \\ -10, & \text{if } d_{i, \text{obs}} \leq 0 \end{cases} \quad (4.16)$$

where D_i^{cover} and D_i^{conn} represent the individual differences in the rewards for user coverage and connectivity, d_{obs} is the distance of the drone i to the closest obstacle or neighbour drone, and $d_{\min}$ defines the minimum separation distance to avoid collisions.

This scheme ensures survival of UAVs by overriding coverage and connectivity objective when safety

margins are violated.

4.2.4 Position and altitude control

The EVSM algorithm provides direct control over the horizontal position and uses a proportional-derivative (PD) controller for altitude hold, as detailed in Section 4.1.5. In contrast, the SDQN algorithm outputs a target horizontal position instead of a force command. To achieve desired position, two PD controllers are employed: one for horizontal position control and another for altitude hold.

The action space includes five basic movement commands: **NOP**, **UP**, **DOWN**, **RIGHT**, and **LEFT**. Each movement action is interpreted as a fixed displacement step of magnitude d applied to the current target position $\mathbf{p}_t$ in the direction $\mathbf{u}_a$ associated with the selected movement action. The **NOP** action and the two zoom-level actions (zoom-in and zoom-out) correspond to a zero-vector direction, resulting in no change in target position. The new target position is updated by

$$\mathbf{p}_{t+1} = \mathbf{p}_t + d \cdot \mathbf{u}_a \quad (4.17)$$

The current simulation runs with a time step of 0.1 s with a displacement step of $d = 2\text{ metres}$. This corresponds to a theoretical maximum speed of 20 m/s , which aligns with the speed limits of UAV models reviewed in Section 3.2.1.

4.2.5 Training process

Training the DQN agent follows the algorithm described in Section 2.4.3. In this case, the environment is multi-agent reinforcement learning (MARL) setup, where each agent uses a central DQN agent during training. However, action processing, state updates, and frame generation are performed independently by each agent. This configuration follows the “centralised training with decentralised execution” paradigm described in [40].

Each training step consists of updating all agents’ states, generating their corresponding observation frames, computing rewards, and storing each agent’s experience tuple in a shared replay buffer. Afterward, a training update is performed using a mini-batch sampled from the replay buffer. Previous frames and actions are stored to compute temporal-difference targets. This process is summarised in Algorithm 4.

Algorithm 4: DQN Training with Prioritized Experience Replay (PER)

Input: Number of agents N , image generator $\mathcal{F}$
Output: Trained Q-network parameters θ and performance metrics
 Initialise Q-network Q_θ , target network Q_{θ^-} , and replay buffer $\mathcal{D}$;
foreach *episode* **do**
 Initialise environment $\mathcal{E}$ with random obstacles and users;
 Initialise states $s_{t-1} \leftarrow s_0$, and frames $\phi_{t-1} \leftarrow \mathcal{F}(s_0)$;
 Set initial actions $a_{t-1} \leftarrow \text{NOP}$;
 foreach *step* t **do**
 Update agents states s_t , rewards r_t and done flags d_t ;
 Generate frames $\phi_t \leftarrow \mathcal{F}(s_t)$;
 foreach *agent* $i = 1 \dots n$ **do**
 Store $(\phi_{t-1}^i, a_{t-1}^i, r_t^i, \phi_t^i, d_t^i)$ in $\mathcal{D}$ with priority $p_t^i = |\delta_t^i| + \varepsilon$;
 end
 if $\mathcal{D}$ contains enough samples **then**
 Sample minibatch $\{(\phi_k, a_k, r_k, \phi_{k+1}, d_k)\}$ from $\mathcal{D}$ using prioritized sampling;
 Set $y_k = \begin{cases} r_k, & \text{for terminal experience: } d_k = \text{True} \\ r_k + \gamma \max_{a \in A} Q_{\theta^-}(\phi_k, a), & \text{for non-terminal experience: } d_k = \text{False} \end{cases}$
 Calculate IS weights w_k with Equation 2.18;
 Compute weighted loss: $\mathcal{L} = \sum_k w_k (y_k - Q_\theta(\phi_k, a_k))^2$ (Eq. 2.16);
 Perform gradient descent step on $\mathcal{L}$ and update θ ;
 Update priorities $p_k \leftarrow |\delta_k| + \varepsilon$ in $\mathcal{D}$;
 Every C steps, update target network: $\theta^- \leftarrow \theta$;
 end
 if *any collision or full users coverage* **then**
 Log metrics and terminate episode;
 end
 For each agent i , select new action a_t^i using ε -greedy policy on $Q_\theta(\phi_t^i, a)$;
 Update previous frames $\phi_{t-1}^i \leftarrow \phi_t^i$, and actions $a_{t-1}^i \leftarrow a_t^i$;
 end
end

Chapter 5

Simulation application

This project involves the development of an UAV simulation tool designed for the testing and validation of distributed swarming algorithms under realistic conditions. The simulator, implemented in Python, includes multiagent simulation, network simulation through NS-3, and graphical visualisation.

All code developed for the simulation tool is available in a public GitHub repository under MIT licence [43]. In addition, the implementation of a custom DQN agent is provided as an external library in [44]. The simulator also relies on NS-3 for network simulation, which is used as an external library available on [45].

5.1 Design process

The development of the simulator followed an iterative design process. First, the system requirements were defined to establish the main functionalities and constraints. Then a preliminary design was proposed, exploring different architectural options and evaluating implementation alternatives. Finally, a final architecture was selected that balances simplicity, modularity, and performance.

5.1.1 System requirements

The simulation tool must be designed to satisfy the following high-level requirements:

- **Multi-agent simulation:** The simulation shall model the dynamics of agents (UAVs, users, and control stations) as well as their interactions with the environment and the simulated network.
- **Realistic network emulation:** The ad-hoc network shall be accurately simulated, including wireless channel modelling, internet stack protocols, and routing mechanisms.
- **Swarming algorithms:** The swarming algorithms described in Section 4 shall be implemented and exhibit the expected behaviours.
- **Reliable inter-process communication:** Reliable and efficient communication mechanisms shall be implemented to enable interaction between the different components (multi-agent sim-

ulator and network simulator).

- **Time synchronization:** Consistent simulation time shall be maintained across all components to ensure coherent state updates.
- **Visualization:** A graphical user interface (GUI) shall provide real-time observation, control, and analysis of the simulation state.
- **Modular architecture:** The software design shall be modular to facilitate maintenance, testing, and future extension.

5.1.2 Preliminary design

The proposed solution is designed to meet the high-level requirements by dividing the simulation tool into three core components, each responsible for specific tasks:

- **Multi-agent simulator:** Models the behaviour and dynamics of agents (drones, users, and control stations) and their interaction with the environment and the emulated network. It handles forwarding of packets generated by users (ingress traffic) to the network simulator, receives outgoing packets (egress traffic) for delivery to the destination agents, and provides current agent positions to the network simulator.
- **Network simulator:** Implemented in C++ using the NS-3 framework [45], it models wireless communications and network protocols to emulate realistic data transmission through the ad-hoc network. It exchanges data packets (ingress and egress traffic) with the multi-agent simulator, obtains current agent positions for accurate path loss modelling, and maintains time synchronisation with the multi-agent simulator.
- **Control application:** Provides a graphical user interface (GUI) for visualising the simulation state and manages the initialisation and operation of all simulator components by sending control commands. It also captures ingress and egress traffic for logging and analysis purposes, facilitating the evaluation of the simulation state and system performance.

Selected technologies

Python was chosen for the multi-agent simulator and control application due to its high-level syntax and emphasis on code readability, which facilitate rapid prototyping and development. Combined with NumPy, it provides efficient numerical operations on large arrays and matrices, making it a powerful framework for developing complex systems with reasonable computational performance.

In contrast, NS-3 is a mature network simulation framework developed in C++. Although NS-3 provides Python support through function bindings, they do not yet provide full compatibility, and simulations must still be executed within NS-3 environment. For this reason, the network simulation module is implemented in C++, taking advantage of native support and facilitating development. A Python-based control script, such as the control application, can then launch the network model within the NS-3 environment and interact with multi-agent simulator through socket communication.

Components interaction

The three main components of the simulator exchange simulation data and control commands through TCP/UDP sockets. The primary advantage of using sockets is the flexibility to implement each component in a different programming language. Moreover, sockets provided standardised interfaces, ensuring compatibility across different development environments and operating systems.

A simple communication API defines the message types and commands exchanged between the two simulators. For simple data transmission, a structured data format like plain text JSON may be used. For larger data, binary serialisation formats like MessagePack or Protobuf will be considered. To simplify communication between components, the use of the ZMQ messaging library will be studied, following the approach proposed in the FlyNetSim article [6].

Table 5.1 summarises the messages sent and received by each of the simulator components.

Table 5.1: Simulator components inputs and outputs

Component	Inputs	Outputs
Multi-agent simulator	Control commands Outgoing packets	Agent positions Incoming packets Sync data
Network simulator	Control commands Agent positions Incoming packets Sync data	Outgoing packets
Control application	Agent positions Ingress/Egress traffic	Control commands

Architecture design

During software development, three architectures were proposed for organising the simulator components. These architectures, listed in order of increasing complexity, are as follows:

1. **Integrated architecture:** In this configuration, the multi-agent simulator and control application are combined within a single Python process. This approach simplifies the interaction of components by using only one socket to communicate with the network simulator, while unifying control logic and keeping the simulation of agents within the same Python environment.
2. **Centralised architecture:** In this approach, the control application acts as an intermediary between the two simulators, serving as a central node for data exchange and simulation control. This architecture requires two sockets to connect each component to the control application. Although this architecture facilitates precise control and synchronisation between components, it may introduce higher latency due to all communication being routed through the central node.
3. **Triangular architecture:** In this design, the three components (multi-agent simulator, network simulator, and control application) are directly interconnected. This reduces communi-

cation latency but requires more complex control logic and presents challenges for maintaining time synchronisation across components.

These three architectures are summarised in Figure 5.1.

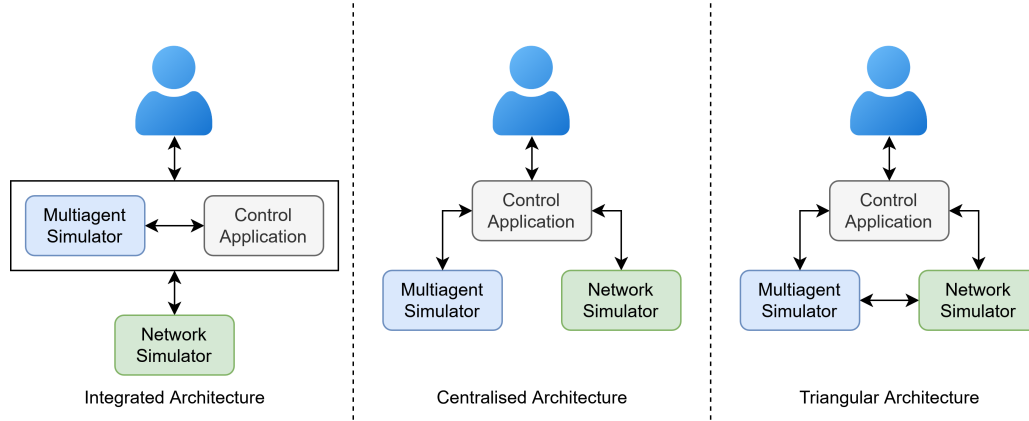


Figure 5.1: Simulator architectures diagrams

5.1.3 Final architecture

After careful consideration, the centralised architecture was selected to simplify code development and control logic. In this design, Python acts as the master process responsible for launching and managing the network simulation. In contrast, NS-3 runs as a slave process and receives commands from multi-agent simulator.

The scope of the control application was reduced to serve as a visualisation tool to display the simulation status, along with a simple command-line logger to assist in debugging and monitoring. All the control logic for launching and maintaining synchronisation between both simulators is embedded within the code modules.

To facilitate communication between simulators, a custom protocol named SimBridge (detailed in Section 5.4) was developed. SimBridge operates as a layer over raw UDP sockets and uses a binary format for message payloads to ensure efficient data transfer. Is implemented in both the network and multi-agent simulator components. The NS-3 SimBridge instance acts as a server, responding to commands and requests sent by the Python client.

5.2 Multi-agent simulator

As introduced in 3.1, the multi-agent simulator is the software component responsible for modelling the behaviour of all agent types involved in the simulation, including drones, users, and control stations. It handles their mobility models, communication policies, and interactions with other components in the system.

5.2.1 Agents overview

The simulation involves three categories of agents, each with distinct mobility models and communication policies, as stated in Table 5.2.

Agent Type	Mobility Model	Connection Policy
Drones	Horizontal 2D movement based on swarming algorithm (EVSM or SDQN). Altitude hold above terrain.	Connect to nearby drones and users. Relay user data and routing messages. Periodically broadcast position to neighbour drones.
Users	2D random walk at ground level (zero altitude).	Connect to near drones and users. Periodically broadcast packets to users in the network.
Control Stations	Fixed ground-level position.	Connect to near drones and users. Record reported positions. Send control commands to UAVs when needed.

Table 5.2: Mobility models and connection policies of each agent type

During the development of the simulator, two design approaches for the implementation of agents were considered:

- **Independent agents:** Each agent is implemented as a standalone object, storing states within class attributes, and updating sequentially by method calls. This approach is intuitive and modular, but computationally inefficient, due to its limited parallelism and redundant calculations.
- **Vectorized agents:** All agents of the same type are represented by a single NumPy array, allowing batch updates and optimised numerical calculations. This method improves performance and scalability but requires careful design considerations to preserve the decentralised behaviour expected for autonomous and independent agents, avoiding the simulation to perform as a centralised system.

The independent agents approach was selected to simplify simulator implementation while focussing parallelisation efforts on other aspects of the code.

5.2.2 Environment

The multi-agent simulator includes an environment module to simulate agent interaction with the surrounding terrain. This module consists of two main components: obstacles and elevation.

The `obstacles` module allows the definition of exclusion zones and boundaries to design the mission area and introduce spatial restrictions. Three types of obstacles are supported: circular, rectangular, and polygonal. Each obstacle can be added to the environment, and a set of methods is provided to check whether a given position lies inside the obstacle, as well as to calculate the distance and

direction to it. These operations are implemented using highly vectorised mathematics and Numba JIT compilation for efficient performance when evaluating large arrays of positions.

A complementary module called **boundaries** was developed to define inverse obstacles, where the valid area is the interior of a shape. This is used to define the outer limits of the mission area.

The **elevation** module provides 3D elevation data for the terrain. The DEM data are loaded from a GeoTIFF file to define the terrain surface within the mission boundary. It includes highly vectorised methods to efficiently retrieve elevation values for large arrays of positions.

This module also implements coordinate transformations between geographic (latitude, longitude, altitude) and local cartesian ENU (East-North-Up) systems, centred at the control station (home position). The conversion assumes a spherical Earth model and is used to translate between global and local reference frames.

GEO to ENU conversion

To simplify conversion from geographic coordinates (latitude, longitude, altitude) to local ENU (East-North-Up), the flat-Earth approximation is used.

Given a reference location (ϕ_0, λ_0, h_0) , also known as “home” point, the ENU coordinates (p_E, p_N, p_U) are calculated as follows:

$$\begin{aligned} p_E &= (\lambda - \lambda_0) \cdot \cos \phi_0 \cdot M \\ p_N &= (\phi - \phi_0) \cdot M \\ p_U &= h - h_0 \end{aligned} \tag{5.1}$$

where ϕ , λ , and h represent latitude, longitude (in degrees), and altitude (in metres), respectively.

The inverse conversion from ENU to geographic coordinates is defined as

$$\begin{aligned} \phi &= \phi_0 + \frac{N}{M} \\ \lambda &= \lambda_0 + \frac{E}{M \cdot \cos \phi_0} \\ h &= h_0 + U \end{aligned} \tag{5.2}$$

where $M \approx 111320$ is the approximate number of meters in one degree of latitude.

This approximation is valid for relatively small regions (e.g., less than 50 km), where the deviation in calculated altitude due to Earth’s curvature is negligible.

5.2.3 Swarming algorithms

The EVSM algorithm runs an individual instance on each agent. To improve performance, different updates for control and spring formation are used, as described in Section 4.1. The implementation relies on highly vectorised maths with NumPy arrays and JIT compilation with Numba for performance leverage. The most computationally expensive parts are spring formation with acute angle test and the calculation of distances and directions to obstacles. This was evaluated using a line profiling tool with 60% of elapsed time in these methods.

In contrast, the SDQN algorithm follows a centralised training and decentralised execution paradigm, as explained in Section 4.2. Each drone interacts with a central DQN agent for frame processing and action selection. This design is more efficient than running independent DQN agents on each drone, because data can be processed in batches, while still maintaining a decentralised behaviour as each drone experiences and actions are independent.

To implement this model, each agent contains an SDQN interface class that is registered in a central agent (SDQN brain), which acts as a proxy between the central DQN agent and each drone interface. When an agent is updated, it generates a new frame, sends it to the interface, and receives a target position. The SDQN brain trains on mini-batches from the replay buffer and selects actions using an epsilon-greedy policy. These actions are then distributed back to the interfaces. This process is shown in Figure 5.2.

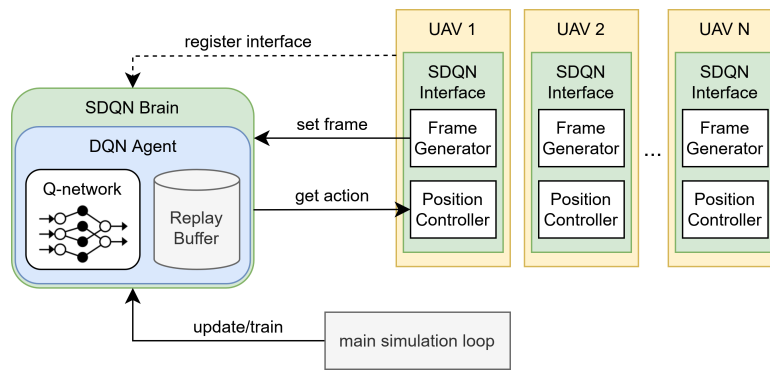


Figure 5.2: SDQN architecture diagram

The DQN agent library used is a custom implementation built with Keras with lower-level TensorFlow optimisations for efficient batch sampling and preprocessing, taking advantage of GPU acceleration with CUDA. This code is available in the repository [44].

5.3 Network simulator

The network simulator is built within the NS-3 network with a simulation file implemented in C++ language. This environment allows for realistic modelling of wireless communication and provides full support for simulating the Internet protocol stack.

NS-3 [45] is a discrete event network simulator designed for Internet systems, used mainly in research and education. It provides detailed models for wireless communication, network protocols, and traffic generation.

Network topology

For network simulation, an NS-3 node is created for each agent in the swarm. All nodes use a wireless channel with a log-distance path loss model with path loss exponent ($n = 2.4$) and are equipped with the IEEE 802.11b Wi-Fi interface that operates in the DSSS modulation mode at 1 Mbps. Additionally, each node includes a full IPv4 stack and uses an ad hoc routing protocol, such as AODV or OLSR, depending on the selected configuration.

Each node is assigned a unique IPv4 address within the 10.0.0.0/16 network, which supports up to 65,534 usable addresses. These addresses follow the format 10.0. x . y , where x denotes the agent type and y the unique ID within that type:

- 10.0.1. y : control stations
- 10.0.2. y : UAVs (drones)
- 10.0.3. y : ground users

These address groupings are not actual subnetworks, but serve as a convention for easier node identification within NS-3. All nodes remain in the same network to ensure proper operation of AODV or OLSR.

5.4 SimBridge protocol

Communication between the multi-agent simulator and the NS-3 network simulator is accomplished by the SimBridge protocol. This protocol was developed to enable interaction and data exchange between the two simulator processes, which are implemented in different programming languages (Python for the multi-agent simulator and C++ for the network simulator). SimBridge operates over raw UDP sockets, defining a set of message types and corresponding commands that are executed when certain packets are received or sent.

The decision to use raw sockets, instead of a messaging library such as ZeroMQ, as proposed in [6], was driven by the desire to simplify the design of the system, following the integrated architecture approach of Section 5.1.2. POSIX sockets, also known as BSD sockets, represent one of the most straightforward methods for inter-process communications (IPC). Given that both simulators run on the same machine, the use of UDP sockets over TCP does not represent a significant risk of packet loss. Although some degree of message loss due to OS network limits or socket buffer overflow is possible, this does not pose a significant risk to the overall operation of the simulator. To mitigate this risk, the SimBridge protocol incorporates a basic timeout mechanism for request-reply commands, ensuring a minimum level of reliability.

5.4.1 Command codes

As in the SwarmLink protocol 3.3.2, SimBridge messages are defined in binary format, with each simulator having to implement an interface to serialise sent packets and deserialise received ones.

The basic structure of these messages contains only two fields: the command code and the payload. The command code is a 1 byte size field that identifies the type of SimBridge message, while the payload corresponds to the rest of the packet bytes.

In the current version of the protocol, there are eleven types of messages which can be grouped into four categories: simulation control commands, network node commands, simulation status requests, and their corresponding replies.

Simulation control commands:

- **Do nothing** (0x00): This command contains no payload and is used to implement a heartbeat mechanism. When a SimBridge interface receives a `DO_NOTHING` message it has to reply with another empty `DO_NOTHING` message.
- **Stop simulation** (0xFF): This command also contains no payload. It is sent by the Python process to terminate the network simulator, making NS-3 to close the IPC socket and stop the event-driven simulator.

Network node commands:

- **Set positions** (0x01): This command is used to update the positions of all nodes in the NS-3 simulation. The payload contains a sequence of node identifiers and their associated 3-dimensional position vectors in metres.
- **Ingress packet** (0x02): This command injects a packet into the NS-3 simulator, emulating ingress traffic generated by an application layer process. The packet is delivered to the specified source node to be transmitted as simulated traffic. The payload includes the source node ID, the source IPv4 address, the destination IPv4 address, and the data to be transmitted. A timestamp is attached to the end of the data field to record the time at which the packet enters the network simulator.
- **Egress packet** (0x03): NS-3 sends this command when a node receives a packet to emulate egress traffic exiting the simulated network. The payload includes the same field as the ingress packet, but with the destination node ID instead of the source. A second timestamp is added at the end of the data field to indicate the time at which the packet leaves the network simulator.

Simulation status request:

- **Request positions** (0xA1): This command is used to query the current positions of all nodes of the network simulator. It contains no payload, and NS-3 must respond with a `REPLY_ALL_POSITIONS` message.
- **Request addresses** (0xA2): This command requests the IPv4 addresses assigned to the network interfaces of all NS-3 nodes. The payload field is empty, and the response should be a `REPLY_ALL_ADDRESSES` message.
- **Request sim time** (0xA3): This command queries the current simulation time of the NS-3 environment. It has no payload, and the network simulator must respond with a `REPLY_SIM_TIME` message.

Simulation status replies:

- **Reply all positions** (0xB1): This message responds to the `REQUEST_POSITIONS` command, containing a list of all node positions. The payload includes a sequence of node IDs and their current 3-dimensional coordinates.
- **Reply all addresses** (0xB2): This is the reply to `REQUEST_ADDRESSES`. The payload contains the node IDs and their associated IPv4 addresses.
- **Reply sim time** (0xB3): This is the reply to `REQUEST_SIM_TIME`. The payload consists of a timestamp with the current simulation time in seconds.

5.4.2 Packet formats

The basic structure of SimBridge messages consists of a single-byte command code followed by a sequence of bytes forming the payload. Based on this structure, five different message formats are defined:

- **Empty packet:** This format applies to the `DO_NOTHING`, `STOP_SIMULATION`, and all request commands. These messages contain no payload, and the packet consists only of the command code.
- **Simulation time:** This is used as response to the `REQUEST_SIM.TIME` command. This message includes a 4-byte payload that represents the current network simulation time. The time is encoded as a 32-bit floating point number in seconds.
- **Node addresses:** This format contains a sequence of node identifiers and their associated IPv4 addresses. Each node ID is represented as an 8-bit unsigned integer, and each IPv4 address is encoded with 4 bytes. The total size of the payload is $5N$ bytes, where N denotes the number of nodes.
- **Node positions:** Similar in structure to the node addresses packet, this message contains a list of node IDs and their corresponding 3-dimensional position. Each coordinate is encoded as a 32-bit floating point number in metres. Each entry occupies 13 bytes (1 byte for the node ID and $3 \cdot 4$ bytes for the coordinates), resulting in a total payload size of $13N$ bytes.
- **Data packet:** This format is used for both ingress and egress packet messages. The payload contains the source or destination node ID (1 byte), the source and destination IPv4 addresses (4 bytes each), the data field of D bytes, and the ingress and egress timestamps (4 bytes each, encoded as 32-bit floating point numbers). The total payload size is $17 + D$ bytes.

These messages encapsulate the SimBridge protocol data that facilitates communication between the Python-based multi-agent simulator and the NS-3 network simulation process implemented in C++. The commands define the application programming interface (API) and the mode of interaction between the two simulators. The following section presents a diagram and an example that illustrates the interaction and operational flow between simulators.

5.4.3 Interaction between simulators

The simulation tool is composed of two main components: a network simulator, built using the NS-3 framework, and a multi-agent simulator, developed in Python. These two components interact through UDP sockets using the SimBridge protocol. NS-3 runs a UDP server on port 9000, while the Python-based simulator acts as a client on port 9001.

The implementation of the SimBridge client and server in both simulators follows the architecture shown in Figure 5.3. This architecture relies on containment, where each component is encapsulated within higher-level functionalities.

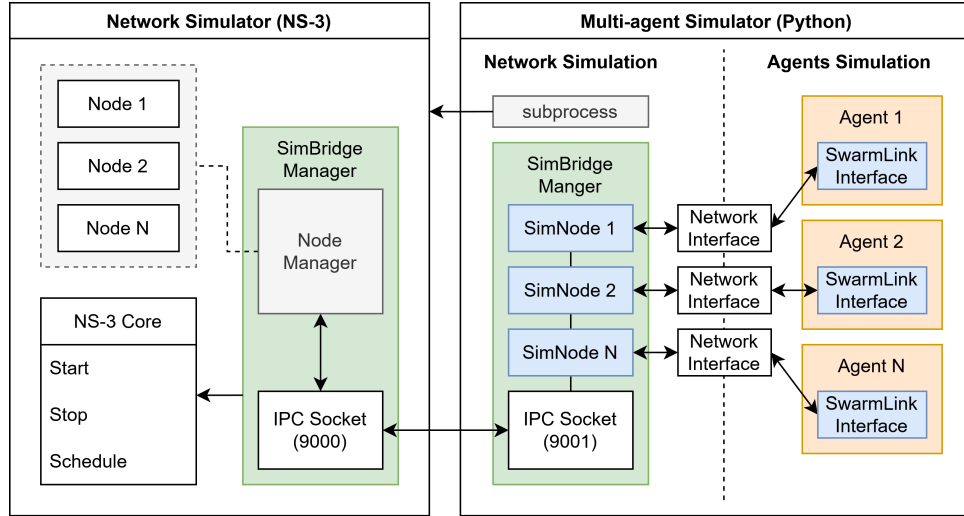


Figure 5.3: SimBridge simulators interaction diagram

5.4.4 NS-3 network simulator

The network simulator operates within the NS-3 environment. NS-3 is an event-driven simulator, and the approach taken to implement SimBridge functionalities was to schedule events for periodic actions or to register callback functions to handle certain events. To facilitate the control of NS-3 network nodes and communication with the Python-based simulator, a wrapper class named SimBridge was developed. This class, which receives the same name as the protocol, processes received commands and decides the actions to be taken, while the low-level communication logic and interaction with the NS-3 simulator are encapsulated in the following components:

- **Node manager** is responsible for the manipulation of the NS-3 nodes. It maintains references to each node and its corresponding socket, providing methods to get and set the position of the node, send data packets, and register the callback function to handle packet reception.
- **IPC socket** encapsulate the logic for setting up, operating, and terminating the UDP socket. The strategy to read the socket is inspired by FlyNetSim [6], in which a periodic polling function is scheduled at the start of the simulation and re-scheduled after each new invocation. The polling interval is currently set to 10 milliseconds.

The `SimBridgeManager` class is responsible for parsing the incoming commands from the IPC socket. When a message is received, the first byte is decoded to determine the command type. Then, `SimBridgeManager` calls the corresponding handler function that deserialises the payload (if any) and performs the associated action. This involves calling an appropriate method from `NodeManager`, such as setting node positions or sending a packet, or executing a NS-3 control function, such as terminating the simulation.

Packet reception within the NS-3 environment is handled by registering a receive callback on each simulated node socket. When a node receives a packet, the callback function encapsulates the payload into a SimBridge data packet and sends it to the Python-based multi-agent simulator.

The NS-3 simulation was configured to run in real-time best-effort mode. This mode synchronises the

NS-3 internal clock with the wall clock time by delaying the simulation when no events are scheduled for the current time. However, when simulation gets overload by scheduled events, such as high frequency packet transmissions, NS-3 may fall behind real time due to lack of processing power.

5.4.5 Python multi-agent simulator

The Python process serves as the central control application and orchestrates both agents and network simulators. A dedicated network simulation class is implemented to manage the NS-3 process and facilitate communication with the SimBridge protocol.

The interaction between multi-agent and network simulators is done through several **NetworkInterface** instances, which act as intermediaries between the SimBridge manager class and each agent's **SwarmLinkInterface**. These interfaces allow agents to send and receive packets without direct interaction with low-level socket code or SimBridge control logic.

The Python network simulator class is responsible for launching and terminating the NS-3 process using the **subprocess** module. The associated SimBridge manager instance contains the IPC socket, which implements all low-level UDP socket control logic, and a collection of **SimNode** data classes to store agent state and buffers for egress data packets.

Socket polling is executed on a separate thread to ensure real-time performance. This allows the multi-agent simulator to run its internal loop at an independent update rate, avoiding socket buffer overflow due to infrequent socket reads.

To synchronise with real-time and NS-3 simulation time, the Python process dynamically adjusts its sleep intervals to match wall-clock time. This was found to be the most reliable approach, as both simulators attempt to maintain real-time synchronisation. In addition, NS-3 is the first simulator to lag behind real-time when increasing the number of nodes. This ensures that both simulators will maintain synchronisation, as the Python process will adjust its delay to wait for NS-3. The Python network simulator acquires the NS-3 time by reading the egress timestamp from the last egress data packet or querying explicitly the simulation time by using the **REQUEST_SIM_TIME** command.

5.4.6 Simulators communication flow

This section explains the main communication flow through the SimBridge protocol between both simulators. The main interactions are the launch of NS-3, termination and exchange of data packet between two agents.

Initialization process

When the Python process starts the simulation, it uses the subprocess library to launch the NS-3 process. The initialisation is as follows:

1. Python looks for previous NS-3 processes and kills them (if any), ensuring a clean startup and no interference between sockets or duplicated NS-3 processes.
2. Python launches the NS-3 simulator as a subprocess by passing the number of each type of

- agent (GCS, drones, and users) as arguments in the command call.
3. Python verifies connection with NS-3 by sending a `DO_NOTHING` command and waiting for the heartbeat response. This is tried up to five times with a timeout of 1 second.
 4. Python verify NS-3 node creation by comparing local `SimNode` data class agent identifier and IPv4 addresses with those in NS-3 nodes using the `REQUEST_NODE_ADDRESSES` command.
 5. Python queries NS-3 simulation time through the SimBridge protocol by sending a `REQUEST_SIM_TIME` command and annotating the initial NS-3 time to calculate the relative elapsed time. This is because previous verification steps introduce a significant delay, and when the agents simulator is launched, the NS-3 process has been running for a few seconds.

To ensure successful initialisation of the network simulator, the Python control process attempts to launch NS-3 a configurable number of times. This is necessary due to the occasional failure of the first initialisation attempts due to unspecified reasons. If all attempts fail, the Python process terminates execution and raises a runtime error.

Position updating

During simulation, agent positions are updated at each simulation step by sending a `SET_NODE_POSITIONS` command that transmits the current positions for all simulated agents. The simulation operates with a time step of 0.01 seconds, providing agents enough time resolution to send and receive data packets at specific intervals.

To ensure synchronisation between the multi-agent simulator implemented in Python and the network simulator in NS-3, periodic verifications are performed. Every second, the Python process sends a `REQUEST_NODE_POSITIONS` command via SimBridge. The returned positions are compared against the locally stored positions in the `SimNode` data classes, which are regularly updated by the multi-agent simulator. This ensures that agents' positions remain consistent across both environments. Minor discrepancies are tolerated, as small deviations in node positions do not significantly affect NS-3 wireless channel modelling.

Termination sequence

To terminate the NS-3 simulator, the Python process performs a shutdown sequence with the following steps:

1. A `STOP-SIMULATION` command is sent through the SimBridge manager, followed by a waiting period of 1 second.
2. The Python application then checks the status of the NS-3 process using `subprocess` module. If NS-3 has terminated correctly, the shutdown is completed successfully.
3. If the NS-3 process has not exited correctly, the Python simulator forces its termination by sending a system-level kill signal.

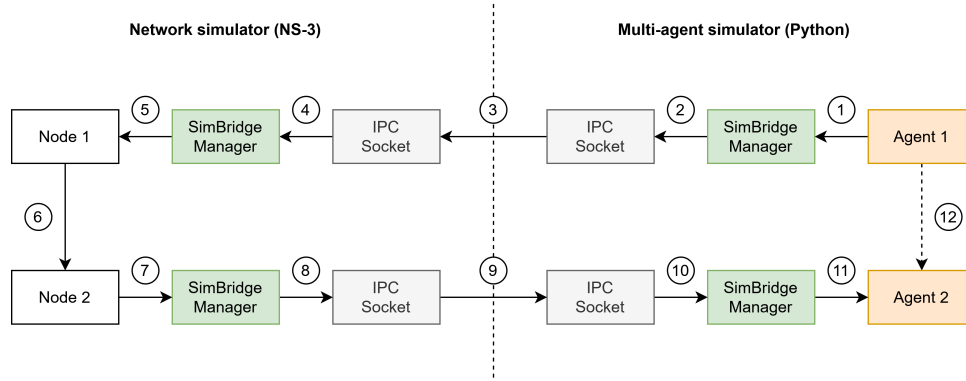


Figure 5.4: Communication flow between simulators with SimBridge protocol

Data packets transmission

Figure 5.4 illustrates an example of communication flow when an agent sends a unicast data packet to another agent. In this scenario, Agent 1 (Node ID = 11, IPv4 = 10.0.3.1) transmits a data package to Agent 2 (Node ID = 12, IPv4 = 10.0.3.2). The process is as follows:

1. Agent 1 initiates transmission by calling the `send()` method of its `SwarmLinkInterface`, specifying the source Node ID (11), source IPv4 address (10.0.3.1), and destination IPv4 address (10.0.3.2). `SwarmLink` protocol encapsulates the packet into a `DATA` packet (see Section 3.3.2 for more details) and sends it through the agent's `NetworkInterface`, which then calls the `send_packet()` method of the `SimBridgeManager`.
2. The `SimBridgeManager` encapsulates the message content into a `SimBridge INGRESS_DATA_PACKET` command and forwards it to the local IPC socket for transmission to the NS-3 simulation.
3. The Python IPC socket sends the UDP packet to its NS-3 counterpart, which receives it and stores its content in the internal buffer.
4. In the next polling cycle in the NS-3 simulation, the `SimBridgeManager` reads all pending packets in the IPC socket and clears its buffer. Then it deserialises the packet and dispatches it to the appropriate handler function based on the command code.
5. The handler function processes the `INGRESS_DATA_PACKET` command, extracting the source Node ID (11), source address (10.0.3.1), destination address (10.0.3.2), and payload. Then, it calls `send_packet()` method from `NodeManager` to send the message through the socket associated with the source node (Node ID = 11) to the destination address.
6. The NS-3 environment performs all network simulation steps such as packet encapsulation, Wi-Fi modulation, channel propagation, noise modelling, etc. The destination node (Node ID = 12) may receive the packet through direct one-hop transmission or via multi-hop forwarding through the ad-hoc network using a routing protocols such as AODV or OLSR.
7. When Node 2 receives the packet the simulated UDP socket triggers a callback function that notifies the NS-3 `SimBridgeManager` to forward the received packet to the Python-based simulator.

8. The `SimBridgeManager` encapsulates the packet into a `EGRESS_DATA_PACKET` command, including the source Node ID, the source and destination IPv4 addresses, and the payload. It then sends this command to the NS-3 IPC socket.
9. The NS-3 IPC socket transmits the packet to the Python socket, which stores it in the internal buffer.
10. When the polling thread reads all pending packets from the socket, it forwards them to the `SimBridgeManager` for processing.
11. The `SimBridgeManager` decodes the `EGRESS_DATA_PACKET` command and delivers the payload to the buffer associated with the destination node (Node ID = 12) within the corresponding `SimNode` data structure. When Agent 2 updates its state, it receives incoming packets from `SwarmLinkInterface`. This calls `NetworkInterface` to retrieve the packet from the appropriate buffer within the `SimBridgeManager`. Then, the `SwarmLinkInterface` extract the payload from the SwarmLink protocol `DATA` packet and returns it to Agent 2.
12. From the agents' perspective, this process appears as a direct message exchange through the `SwarmLinkInterface`, abstracting all underlying network simulation process. The total round-trip delay is mainly determined by NS-3 network simulation, as IPC socket transmission and `SimBridge` processing times are negligible.

For the SwarmLink protocol automatic position reporting service, the communication process follows the same steps up to step 6. In this case, since the `POSITION` packet is broadcast, multiple nodes may receive it, resulting in several delivery operations to Python-based simulator.

5.5 Visualization applications

To facilitate analysis and debugging of the simulation, multiple visualisation tools were developed. These tools provide both 2D and 3D representations of the simulation state, as well as a representation of the network topology. Visualisations enable real-time tracking of agents, obstacles, connectivity, and signal quality.

2D simulation viewer

The 2-dimensional simulation viewer, implemented using `matplotlib`, displays the horizontal position of all agents, including UAVs, ground users, and control stations. It also renders obstacles, the mission area boundary, and a configurable background image. This background can be configured to show a satellite image of the terrain, an elevation map, or a heatmap of the RSSI (Received Signal Strength Indicator) signal quality, as shown in Figures 5.5 and 5.6.

Three variants of the 2-dimensional viewer were developed:

- **Base version:** A general-purpose simulation visualiser extended by other algorithm-specific viewers. It displays all agents, obstacles, and the selected background layer.
- **EVSM version:** Extends the base viewer by highlighting edge UAVs in a different colour and displaying the spring connections as lines between drones.

- **SDQN version:** Highlights the first drone in red and visualises its perception frames in real time, as described in Section 4.2.

Figure 5.5 (left) illustrates the EVSM viewer: interior UAVs are represented by blue crosses, edge UAVs by red crosses, users by magenta dots, and the control station by a black star. Obstacles are represented as red polygons, and the boundary of the environment is shown as a red line. The background image displays an RSSI heatmap scaled from 0% to 100%.

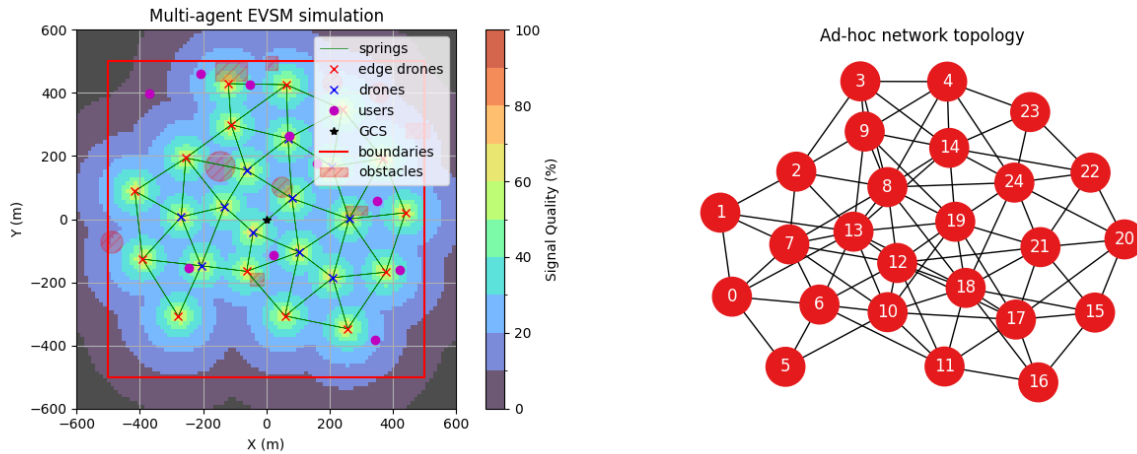


Figure 5.5: EVSM simulation (left) and network (right) viewers.

Network topology viewer

To represent the wireless network structure, the **NetworkX** Python module is employed. This module constructs and displays the network topology, identifies connected components, and visualises UAV clusters in different colours if needed, as illustrated in Figure 5.5 (right).

At each simulation step, a connectivity matrix is generated by checking which drones receive non-zero RSSI values from others. This matrix is used to define the adjacency relations in the graph, which is then rendered using **NetworkX** and **matplotlib**.

It should be noted that the network topology viewer in Figure 5.5 represents actual communication links calculated from RSSI measurements and should not be confused with the spring connections shown in the EVSM visualisation.

3D terrain visualisation

For a realistic simulation scenario, we model the environment as a 3D surface obtained from an elevation map covering a 10 km by 10 km area (100 km²) from the city of Barcelona. Elevation data was obtained from **OpenTopography** [46], providing a detailed 3D model of the terrain. The satellite image for the same bounding box was obtained using the **contextily** library [47] for Python. Both the elevation heat-map and the satellite view are shown in Figure 5.6.

The geographic limits for the scenario correspond to the coordinates listed in table 5.3.

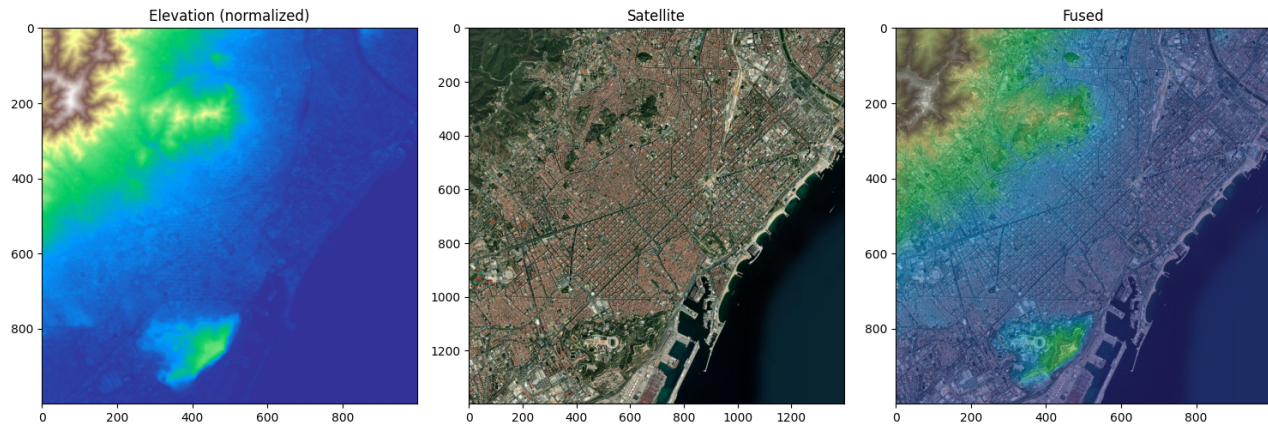


Figure 5.6: Elevation heatmap (left), satellite image (centre), and fused image (right) of mission area.

Table 5.3: Bounding box coordinates for simulated scenario.

	West	South	East	North
Value	2.1099°	41.3501°	2.2299°	41.4401°

Real-time 3D rendering is not efficiently supported by `matplotlib`. In addition, it does not handle occlusion correctly when overlaying 3D surfaces. To address this, the 3-dimensional terrain visualisation module was implemented using `PyVista`, as illustrated in Figure 5.7.

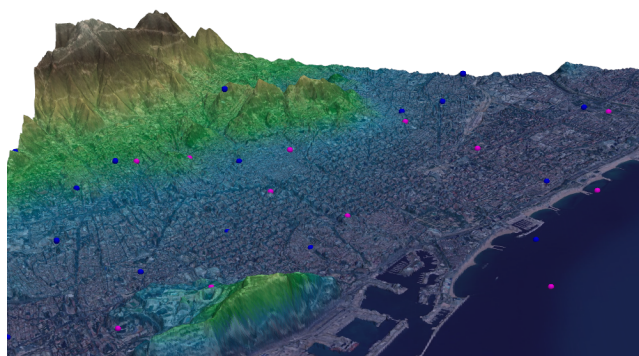


Figure 5.7: PyVista 3D terrain visualizer

The terrain viewer uses digital elevation model (DEM) data from the elevation module, overlaying it with a satellite image and a terrain heatmap based on altitude. Agents are plotted as coloured spherical points: blue for drones, magenta for users, and red for the control station. The Qt interface allows users to interactively control the camera view, including zooming, panning, and rotating the scene to explore the terrain and drone positions in detail.

Chapter 6

Simulation Results

This chapter presents a series of tests conducted to evaluate both the implemented swarming algorithm and the hybrid simulation framework (multi-agent and network simulation). First, we analyse network simulation performance for different number of agents. Next, performance metrics for SDQN training process are discussed. Finally, the impact of position information accuracy on EVSM performance is examined by contrasting results using ideal position data with those obtained via the SwarmLink protocol, which incorporates network simulation with realistic delay and packet loss.

6.1 Network simulator performance

To evaluate network simulation performance, a series of tests were conducted in the NS-3 framework with different numbers of agents. The scenario consisted of two ground users exchanging data packets through the ad-hoc network formed by UAVs. The objective was to evaluate the average transmission time, the delay introduced by inter-processes communication, and the real-time restrictions of NS-3.

The test scenario consisted of a grid of $N \times N$ drones flying at 50.0 metres of altitude, with User 1 placed at the origin and User 2 placed at the opposite corner. During simulation, User 1 sends data packets to User 2 each 100 ms over a 100 seconds simulation, sending a total of 1,000 packets. The spacing between drones ensured that only adjacent nodes were within communication range, enforcing a multi-hop routing pattern.

The AODV routing protocol was selected because of its faster convergence compared to OLSR, despite its higher control overhead. In addition to data packets exchange by users, AODV introduces additional network traffic through frequent control messages for neighbour discovery and route maintenance.

Performance metrics

To measure network simulator performance, four metrics were calculated: average packet loss, delay between real-time and NS-3 simulation time, NS-3 round-trip time (RTT) within network simulation, and global RTT including inter-processes communication delays.

The performance metrics were recorded as follows:

1. The multi-agent simulator transmitted one packet every 100 ms from User 1 to User 2. Each packet is marked with a unique identifier and a real-time timestamp.
2. Packets were delivered to the NS-3 simulator through the SimBridge interface and marked with NS-3 simulation time (ingress timestamp), as described in Section 5.4.
3. Within the NS-3 environment, packets were sent by User 1 and forwarded through the ad hoc network. When packet is received by User 2 its marked with egress timestamp.
4. Packets were sent back to multi-agent simulator, where real-time and NS-3 timestamps were logged.

The tests were conducted with 4, 9, 16, 25, 36, and 49 UAVs, each repeated 10 times to obtain average values. The resulting metrics are summarised in Figure 6.1, with detailed values provided in Table 6.2.

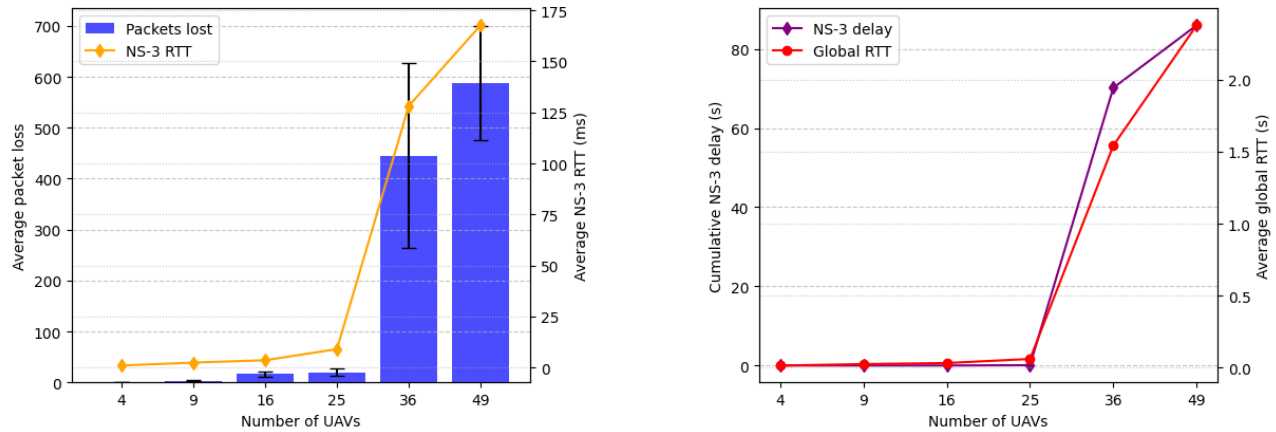


Figure 6.1: Network simulator test results

The following metrics were used to evaluate network simulation performance:

- **NS-3 Delay:** The difference between real time and NS-3 simulated time after 100 seconds. This metric reflects how closely NS-3 follows real time. NS-3 begins to lag beyond 25 UAVs, causing the multi-agent simulator to slow down to maintain synchronisation.
- **Global Round-Trip Time (RTT):** Measured from the time a packet is sent to when it is received, using real-time clocks in the multi-agent simulator. This includes simulated transmission delays and inter-process communication overhead. Global RTT is typically about ten times greater than NS-3 RTT, indicating higher perceived delay from the agents' perspective.
- **NS-3 Round-Trip Time (RTT):** The time difference between ingress and egress timestamps within NS-3. It reflects internal network congestion and scales independently of real time. RTT increases significantly beyond 25 UAVs, likely due to processing overhead and interface congestion.
- **Packet Loss:** Mainly caused by buffer overflows in the simulated network. Additional loss may occur above 36 UAVs due to NS-3 lag and less frequent polling of IPC sockets.

Results discussion

These results show that simulation runs in real-time with up to 25 UAV nodes. Beyond this point, NS-3 begins to lag, forcing the multi-agent simulator to slow down in order to maintain synchronisation, resulting in a synchronised but slower than real-time simulation. The simulation environment was set up on a Windows 11 PC with specifications summarized in Table 6.1.

Table 6.1: PC specifications for DQN training

Component	Specification
Operating System	Windows 11
Processor	AMD Ryzen 5 5600X, 6 cores @ 3.7 GHz
RAM	32 GB DDR4
Graphics Card	NVIDIA GeForce RTX 3060 Ti

Simulating more than 49 UAVs is not feasible due to UDP packet size limitations with the SimBridge sockets, causing the `SET_NODE_POSITIONS` command to fail. Although technically possible, simulations with more than 25 UAVs were not performed due to excessive delays and performance degradation. Future improvements could include optimising interprocess communication and increasing available computational resources.

Future optimisation strategies include enhancing the interprocess communication mechanism and expanding computational resources to support larger-scale simulations.

Table 6.2: Network simulator test results

Number of UAVs	4	9	16	25	36	49
Real time (s)	100.01	100.01	100.01	100.01	100.02	100.10
NS-3 time (s)	100.02	100.03	100.03	99.94	29.83	14.11
Packets lost avg	0.00	3.50	16.83	20.22	445.00	587.50
Packets lost std	0.00	0.58	5.42	7.58	181.58	111.91
RTT global avg (s)	0.0136	0.0234	0.0308	0.0592	1.5428	2.3786
RTT global min (s)	0.0037	0.0108	0.0154	0.0271	0.0501	0.1171
RTT global max (s)	0.0310	0.4123	0.7330	1.9000	13.7156	9.0544
RTT global std (s)	0.0040	0.0172	0.0339	0.1043	1.5468	1.7763
RTT NS-3 avg (s)	0.0010	0.0024	0.0036	0.0090	0.1282	0.1677
RTT NS-3 min (s)	0.0010	0.0022	0.0024	0.0035	0.0049	0.0075
RTT NS-3 max (s)	0.0103	0.0581	0.4217	1.4669	1.4191	1.1285
RTT NS-3 std (s)	0.0003	0.0023	0.0191	0.0720	0.1401	0.1536

6.2 SDQN training

This section studies the training performance of the SDQN algorithm and evaluates whether the expected objectives are achieved. The training environment comprises a square mission area of $2\text{ km} \times 2\text{ km}$ with 16 drones initially positioned at the centre in a 4×4 grid formation with a 10 m separation. For each episode, 20 users are randomly placed inside the area, as well as 4 obstacles with random

positions and sizes. An episode is terminated if a drone collides, or if users coverage reaches 100%. Episodes are limited to a maximum of 5,000 steps, with a time step of 0.1 seconds, resulting in a maximum duration of 500 seconds. These hyperparameters used during the training are summarised in Table 6.3.

Table 6.3: SDQN Training Parameters

Parameter	Value
Time step δt	0.1 s
Number of agents	16
Mini-batch size	64
Discount factor γ	0.99
Learning rate (Adam optimizer)	$1 \cdot 10^{-4}$
Loss function	MSE
Epsilon initial value ε_0	1.0
Epsilon decay (linear) $\Delta\varepsilon$	$-1 \cdot 10^{-5}$
Epsilon min value ε_{min}	0.1
Replay buffer size	$200 \cdot 10^3$ experiences
PER priority exponent α	0.4
PER weight correction β	0.6
PER β annealing	$+1 \cdot 10^{-6}$ per step
Target network update frequency	Every 10,000 steps
Max episode duration	5,000 steps
Training area size	$2 \text{ km} \times 2 \text{ km}$
Number of users	20
Random obstacles	4 with 100 m to 1.0 km size

The reward function of Equation 4.16 was modified by removing the connectivity reward, because the drones remained stuck in the initial grid formation to maximise it. Only the user coverage reward was used. This results in drones learning first to spread out to avoid collision penalties and then start focussing on maximising user coverage. Although good user coverage is achieved, this leads to a network that is not fully connected, as previously discussed in Section 4.2.3.

Training performance metrics for 160 episodes are presented in Figure 6.2 (left). The mean loss per episode decreases rapidly during the initial episodes, reaching around 10^{-4} after 160 episodes. The cumulative reward per episode, calculated as the total reward divided by the number of agents, rises rapidly from approximately -500 to oscillate between 0 and 200, indicating a fast but unstable learning.

Figure 6.2 (right) illustrates the user coverage, area coverage, and global connectivity rates achieved at the end of each episode. These curves are smoothed using a 10-episode moving average to facilitate visualisation. In the last 60 episodes, the user coverage reaches 80-90%, while the area coverage remains near 60%, demonstrating that the system can learn to target user locations without covering the entire mission area. However, global connectivity drops to around 40%, remarking the impact of excluding the connectivity reward.

Overall, SDQN performance is lower compared to the EVSM algorithm. Although SDQN operates in a larger area with fewer UAVs than EVSM, similar performance was expected to that observed in Section 6.3, as SDQN focusses on covering users, not the entire area. Each episode simulates up to 8

minutes (50,000 steps), with some episodes terminating early due to collisions. In the best cases, SDQN reaches 90% user coverage at the end of the full episode duration, and in many cases, coverage remains lower. This is significantly below expectations, as the target was to reach full coverage within 1–2 minutes, as achieved by EVSM in equivalent scenarios. As a result, a direct performance comparison between SDQN and EVSM is not presented.

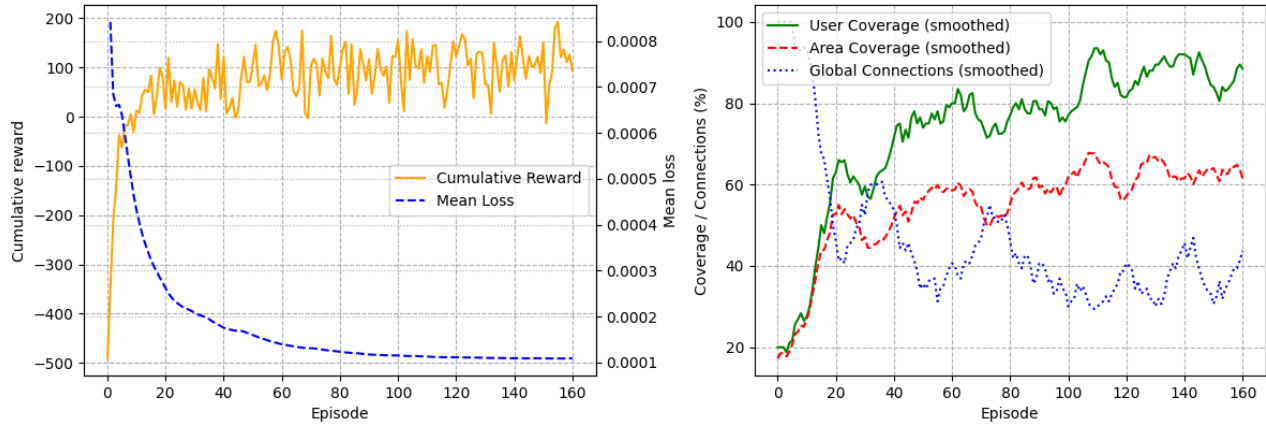


Figure 6.2: Training performance of the SDQN algorithm

The 160 training episodes required approximately 16 hours to complete, with a mean duration of 6 minutes per episode. During training, a total of 800,000 Q-network gradient steps were performed. These values are due to available computational resources, as detailed in Table 6.1. Increasing processing power could significantly reduce training time.

6.3 Ideal vs network-based simulation

To assess the impact of realistic network simulation on system performance, two tests were conducted on UAV deployment with the EVSM algorithm. The test scenario comprises a $1\text{ km} \times 1\text{ km}$ area containing 3 obstacles and 25 drones arranged in a grid formation 5×5 , as described in the scenario in Figure 4.1. In addition, 10 ground users are randomly placed within the mission area to verify the communication capability of the users.

In the first simulation, referred to as the ideal case, each UAV obtains the positions of its neighbouring drones directly from the simulator, assuming instantaneous and error-free communication. In the second simulation, neighbour position information is acquired through the SwarmLink protocol over network simulated in NS-3, introducing realistic communication delays and packet loss. The EVSM was selected because the SDQN algorithm cannot provide a fully connected network, making it unsuitable for operation under realistic network conditions.

The results, illustrated in Figure 6.3, show that full area and user coverage is achieved in both cases. However, in the ideal case, the maximum area and user coverage is reached faster than in realistic network simulation, about 10 and 20 seconds earlier, as summarised in Table 6.4. This reduction in EVSM performance is caused by the lag in received position information and packet loss during periods of network congestion.

The second simulation also serves to validate the user's ability to send and receive messages under

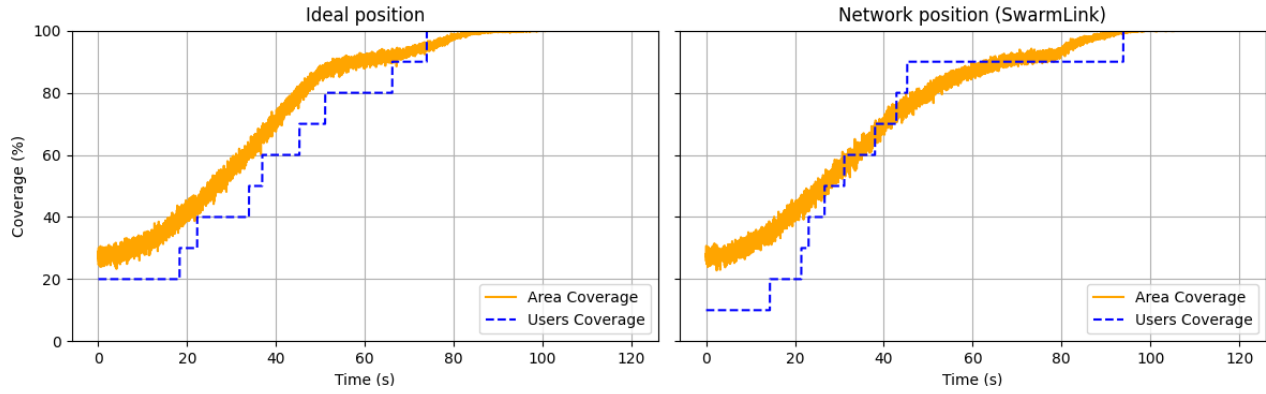


Figure 6.3: EVSM coverage performance with ideal and network-based position data

Table 6.4: Time to reach full area and user coverage

Simulation type	Area coverage time (s)	User coverage time (s)
Ideal (direct positions)	84.65	73.91
Network-based (SwarmLink)	93.52	93.89

network constraints. On average, each user transmitted 25.0 packets and successfully received 24.2 packets, indicating that only 3.2% of the data packets were lost. This low loss rate does not significantly affect system performance, as users can retransmit lost packets, demonstrating the feasibility of the system under realistic network simulation.

Chapter 7

Conclusions

This chapter summarises the knowledge acquired through research on swarming algorithms for UAVs and deep reinforcement learning techniques. It also evaluates the hybrid multi-agent and network simulation tool developed. The following sections review the results obtained, the limitations of the chosen approach, and potential areas for future improvement.

7.1 Project results

The project work effort can be divided into two main areas: study and implementation of decentralised swarming algorithms for UAV networks and the development of the hybrid simulation tool combining multi-agent and network simulator. The first area includes both the EVSM algorithm and the proposed SDQN algorithm based on deep reinforcement learning.

Modified EVSM algorithm

In Section 4.1 we proposed a modified version of the original EVSM algorithm [1] with performance optimisations and an improved collision avoidance mechanism. Its effectiveness in providing coverage throughout the mission area and maintaining a fully connected network was successfully demonstrated in Section 6.3 in both ideal and realistic scenarios.

These results highlight how this well-established algorithm can effectively place UAVs as aerial base stations (ABSs). Although a performance decrease was observed for complex scenarios, such as urban areas, the guaranteed convergence of EVSM makes it a reliable option when selecting swarming algorithms for UAV deployment.

It is also worth highlighting the implementation of a simple protocol called SwarmLink (Section 3.3), developed to enable inter-users communication and facilitate the exchange of UAV position information.

DRL application attempt

During this work, an in-depth study of deep reinforcement learning (DRL) techniques and their applicability to UAV networks was carried out. A DQN-based swarming algorithm was proposed, known as SDQN (Swarming Deep Q-Network).

A custom implementation of the DQN agent with prioritised experience replay (PER) was developed to accelerate the learning process. This was validated against classical Atari games, as in the original DeepMind article [16], demonstrating expected behaviour in general testing environments.

The SDQN algorithm took many design iterations to refine its capabilities. Various frame generators were explored to encode local environment information, proposing an advance log-polar representation to capture both near and distant environment features.

The most critical aspect was designing an effective reward scheme for multi-agent reinforcement learning (MARL) systems. To address this, a difference reward approach was proposed in Section 4.2.3 as proposed in [40] to be implemented, encouraging individual agents to achieve global objective from individual rewards. However, integrating both full user coverage and global network connectivity objectives proved challenging, leading to the exclusion of the connectivity condition from the final reward function.

Despite extensive training using various hyperparameters and evaluation in a simulated environment, the SDQN approach did not provide satisfactory results. Nevertheless, this work supplies a comprehensive framework for further development of deep learning techniques for multi-agent aerial systems. The custom DQN library and simulation tool constitute the foundation for future work which may ultimately lead to an effective design of the SDQN algorithm.

Hybrid simulation tool

A multi-agent Python-based simulation tool, complemented by the NS-3 network simulator, was developed for UAV networks research. The results presented in Section 6.1 demonstrate the capability of the application to simulate up to 49 agents in real-time or with controlled slow-time. This simulation environment served as the primary platform for developing and validating the swarming algorithms presented in this work, and it offers a comprehensive framework for future research and evaluation of UAV network systems.

7.2 Future works

Despite some limited results obtained, UAV networks and swarming algorithms remain a promising research area with many applications. The unsatisfactory results of SDQN call for exploring alternative implementations. In addition, while the hybrid simulator provides an useful tool for UAV network research, improvements can be studied to leverage its capabilities. The following future work and areas of improvement are proposed:

- The SDQN algorithm did not provide the expected results. Two main lines for improvement are proposed: the reward scheme and the model architecture. Designing an effective reward scheme proved challenging, particularly as the connectivity condition was omitted. To address

this, value-decomposition networks for individual reward approximation could be explored [48] as an alternative to difference rewards in MARL environments. Additionally, the existing model architecture based on convolutional neural networks (CNN) may be limiting. More advanced architectures, such as long short-term memory networks (LSTMs) or transformers, could be studied to improve DQN agent capabilities.

- The current UAV simulation uses simplified point-mass dynamics. Future work could incorporate full quad-rotor and fixed-wing models, including aerodynamic and flight dynamics. This would require integrating an autopilot system for attitude control and navigation with realistic noisy sensor inputs. Additionally, modelling actuator dynamics such as DC motors or servo-controllers and battery consumption would improve simulation fidelity.
- The current simulator architecture relies on a central Python process with an NS-3 subprocess. Alternative architectures could be explored, such as decoupling simulator components into separate containers using Docker. Additionally, the existing SimBridge protocol uses raw UDP sockets with commands limited by packet size. Replacing this with a gRPC-based API would simplify inter-process communication and provide more reliable and structured messaging protocol. This approach is commonly used in container-based distributed systems.
- The multi-agent simulator only supports position control through swarming algorithms. To simulate a more realistic system, different UAV operating modes should be implemented, such as deploy, swarm, return to home, and recharge. These modes would be controlled by the control station by sending commands over the ad-hoc network. Additionally, the current SwarmLink protocol only supports position reporting and messaging. New command types and an acknowledgement mechanism should be developed to improve communication reliability.
- The current visualisation application is not interactive. A more advanced GUI could be implemented using Qt to emulate real system interface capabilities, allowing the user to define mission area, monitor detailed information of UAVs status, and send control commands through the control station.
- Finally, the NS-3 network simulator operates in real-time or delayed-time mode. Implementing accelerated-time simulation in NS-3 could significantly reduce experimentation time, although this would require a more complex synchronisation mechanism with the multiagent simulator.

These areas of improvement can be explored to enhance the simulator's capabilities or to develop a functional version of the SDQN algorithm that provides satisfactory results.

Glossary

Acronym	Definition	Acronym	Definition
A2G	Air-To-Ground	ABS	Aerial Base Station
AI	Artificial Intelligence	AODV	Ad hoc On-Demand Distance Vector
AP	Access Point	ANN	Artificial Neural Network
AR	Aerial Relay	CNN	Convolutional Neural Network
D2D	Device-To-Device	DL	Deep Learning
DNN	Deep Neural Network	DQN	Deep Q-Network
DRL	Deep Reinforcement Learning	DSSS	Direct Sequence Spread Spectrum
EPC	Evolved Packet Core	EVSM	Extended Virtual Spring Mesh
FANET	Flying Ad-hoc Network	GCS	Ground Control Station
GU	Ground User	HAP	High-Altitude Platform
HWMP	Hybrid Wireless Mesh Protocol	IBSS	Independent Basic Service Set
IPC	Inter-Process Communication	IS	Importance Sampling
LAP	Low-Altitude Platform	LTE	Long-Term Evolution
LSTM	Long Short-Term Memory	MDP	Markov Decision Process
ML	Machine Learning	MARL	Multi-Agent Reinforcement Learning
OLSR	Optimised Link State Routing	PD	Proportional Derivative
RL	Reinforcement Learning	ROS	Robot Operating System
RSSI	Received Signal Strength Indicator	SAR	Search And Rescue
SDQN	Swarming Deep Q-Network	SITL	Software In The Loop
STA	Station (Wi-Fi mode)	TD	Temporal Difference
TDD	Time Division Duplex	UABS	Unmanned Aerial Base Station
UAV	Unmanned Aerial Vehicle	UE	User Equipment
UDP	User Datagram Protocol	VSM	Virtual Spring Mesh
WLAN	Wireless Local Area Network		

Bibliography

- [1] K. W. Derr and M. Manic, “Extended virtual spring mesh (EVSM): The distributed self-organizing mobile ad hoc network for area exploration,” *IEEE Transactions on Industrial Electronics*, vol. 58, pp. 5424–5437, 2011.
- [2] W. Zhao, S. Cui, W. Qiu, Z. He, Z. Liu, X. Zheng, B. Mao, and N. Kato, “A survey on DRL based UAV communications and networking: DRL fundamentals, applications and implementations,” *ArXiv*, vol. abs/2502.12875, 2025.
- [3] J. Qiu, J. Lyu, and L. Fu, “Placement optimization of aerial base stations with deep reinforcement learning,” *ICC 2020 - 2020 IEEE International Conference on Communications (ICC)*, pp. 1–6, 2019.
- [4] P. V. Klaine, J. P. B. Nadas, R. D. Souza, and M. A. Imran, “Distributed drone base station positioning for emergency cellular networks using reinforcement learning,” *Cognitive Computation*, vol. 10, pp. 790 – 804, 2018.
- [5] Y. Zeng, Q. Wu, and R. Zhang, “Accessing from the sky: A tutorial on UAV communications for 5G and beyond,” *Proceedings of the IEEE*, vol. 107, pp. 2327–2375, 2019.
- [6] S. H. Baidya, Z. Shaikh, and M. Levorato, “FlyNetSim: An open source synchronized uav network simulator based on ns-3 and Ardupilot,” *Proceedings of the 21st ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems*, 2018.
- [7] M. Mozaffari, W. Saad, M. Bennis, Y.-H. Nam, and M. Debbah, “A tutorial on UAVs for wireless networks: Applications, challenges, and open problems,” *IEEE Communications Surveys Tutorials*, vol. 21, pp. 2334–2360, 2018.
- [8] N. I. Sarkar and S. Gul, “Artificial intelligence-based autonomous UAV networks: A survey,” *Drones*, 2023.
- [9] H. Shakhatreh, A. H. Sawalmeh, A. Al-Fuqaha, Z. Dou, E. Almaita, I. Khalil, N. S. Othman, A. Khreishah, and M. Guizani, “Unmanned aerial vehicles (UAVs): A survey on civil applications and key research challenges,” *IEEE Access*, vol. 7, pp. 48 572–48 634, 2019.
- [10] United Nations, “Sustainable development goals,” <https://sdgs.un.org/goals>, 2015, accessed: 2025-04-28.
- [11] A. Zanoletti, E. Carena, C. Ferrara, and E. Bontempi, “A review of lithium-ion battery recycling: Technologies, sustainability, and open issues,” *Batteries*, 2024.

- [12] A. Merwaday and I. Güvenç, “UAV assisted heterogeneous networks for public safety communications,” *2015 IEEE Wireless Communications and Networking Conference Workshops (WCNCW)*, pp. 329–334, 2015.
- [13] A. I. Hentati and L. Chaari, “Comprehensive survey of UAVs communication networks,” *Comput. Stand. Interfaces*, vol. 72, p. 103451, 2020.
- [14] Y. Bu, Y. Yan, and Y. Yang, “Advancement challenges in UAV swarm formation control: A comprehensive review,” *Drones*, 2024.
- [15] B. Shucker and J. K. Bennett, “Virtual spring mesh algorithms for control of distributed robotic macrosensors ; cu-cs-996-05,” University of Colorado, Tech. Rep., 2005. [Online]. Available: <https://scholar.colorado.edu/concern/reports/z603qz26b>
- [16] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. A. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [17] K. W. Derr and M. Manic, “Wireless sensor network configuration—part ii: Adaptive coverage for decentralized algorithms,” *IEEE Transactions on Industrial Informatics*, vol. 9, pp. 1728–1738, 2013.
- [18] IBM, “Machine learning,” n.d., accessed: 2025-04-28. [Online]. Available: <https://www.ibm.com/think/topics/machine-learning>
- [19] M. Hu, J. Zhang, L. A. Matkovic, T. Liu, and X. Yang, “Reinforcement learning in medical image analysis: Concepts, applications, challenges, and future directions,” *Journal of Applied Clinical Medical Physics*, vol. 24, 2022.
- [20] IBM, “What is deep learning?” <https://www.ibm.com/think/topics/deep-learning>, 2020, accessed: 2025-05-01. [Online]. Available: <https://www.ibm.com/think/topics/deep-learning>
- [21] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [22] V. François-Lavet, P. Henderson, R. Islam, M. G. Bellemare, and J. Pineau, “An introduction to deep reinforcement learning,” *Found. Trends Mach. Learn.*, vol. 11, pp. 219–354, 2018.
- [23] S. Thrun, “Efficient exploration in reinforcement learning,” 1992.
- [24] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, 1st ed. USA: John Wiley & Sons, Inc., 1994.
- [25] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, “Playing atari with deep reinforcement learning,” *ArXiv*, vol. abs/1312.5602, 2013.
- [26] L. Lin, “Self-improving reactive agents based on reinforcement learning, planning and teaching,” *Machine Learning*, vol. 8, pp. 293–321, 1992.
- [27] H. V. Hasselt, “Double q-learning,” in *Neural Information Processing Systems*, 2010.
- [28] H. V. Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double q-learning,” in *AAAI Conference on Artificial Intelligence*, 2015.

- [29] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” *CoRR*, vol. abs/1511.05952, 2015.
- [30] W. Wang, Y. Liu, R. Srikant, and L. Ying, “3M-RL: Multi-resolution, multi-agent, mean-field reinforcement learning for autonomous UAV routing,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, pp. 8985–8996, 2021.
- [31] F. Pasandideh, J. P. C. L. da Costa, R. Kunst, N. Islam, W. Hardjawana, and E. P. de Freitas, “A review of flying ad hoc networks: Key characteristics, applications, and wireless technologies,” *Remote. Sens.*, vol. 14, p. 4459, 2022.
- [32] F. Noor, M. A. Khan, A. Y. Al-Zahrani, I. Ullah, and K. A. Al-Dhlan, “A review on communications perspective of flying ad-hoc networks: Key enabling wireless technologies, applications, challenges and open research topics,” *Drones*, 2020.
- [33] A. S. Abdalla and V. Marojevic, “Communications standards for unmanned aircraft systems: The 3GPP perspective and research drivers,” *IEEE Communications Standards Magazine*, vol. 5, pp. 70–77, 2021.
- [34] IWAVE Communications Co., Ltd., “Patron-x10 airborne LTE base station for emergency rescue and disaster response,” <https://www.iwavecomms.com/>, 2023, accessed: 2025-05-26.
- [35] E. A. Marconato, M. Rodrigues, R. de Melo Pires, D. F. Pigatto, L. C. Q. Filho, A. R. Pinto, and K. C. Branco, “AVENS - a novel flying ad hoc network simulator with automatic code generation for unmanned aircraft system,” in *Hawaii International Conference on System Sciences*, 2017.
- [36] N. R. Zema, A. Trotta, E. Natalizio, M. D. Felice, and L. Bononi, “The CUSCUS simulator for distributed networked control systems: Architecture and use-cases,” *Ad Hoc Networks*, vol. 68, pp. 33–47, 2018.
- [37] J. Ponniah, O. D. Dantsker, and R. Mancuso, “Design of multi-agent UAV simulator to support the development of the marsnet communication protocol,” *AIAA Aviation 2019 Forum*, 2019.
- [38] R. W. Beard and T. W. McLain, “Small unmanned aircraft: Theory and practice,” 2012.
- [39] L. A. Remmelzwaal, A. K. Mishra, and G. F. R. Ellis, “Human eye inspired log-polar pre-processing for neural networks,” *2020 International SAUPEC/RobMech/PRASA Conference*, pp. 1–6, 2019.
- [40] J. N. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, “Counterfactual multi-agent policy gradients,” in *AAAI Conference on Artificial Intelligence*, 2017.
- [41] D. H. Wolpert and K. Tumer, “Optimal payoff functions for members of collectives,” *Adv. Complex Syst.*, vol. 4, pp. 265–280, 2001.
- [42] R. E. Tarjan, “Depth-first search and linear graph algorithms,” *SIAM J. Comput.*, vol. 1, pp. 146–160, 1972.
- [43] P. R. Escudero, “UAV swarm simulator,” <https://github.com/pabloramesc/uav-swarm-sim>, 2025, code repository. Accessed: 2025-05-30.
- [44] —, “DQN-lab: custom DQN agent library,” <https://github.com/pabloramesc/dqn-lab>, 2025, code repository. Accessed: 2025-05-30.

- [45] ns-3 project, “ns-3 Network Simulator,” <https://www.nsnam.org/>, accessed: 2025-04-27.
- [46] OpenTopography, “Opentopography,” 2025, accessed: 2025-04-13. [Online]. Available: <https://www.opentopography.org/>
- [47] D. Darribas and C. contributors, “Contextily: a python library to add basemaps to static maps,” 2025, accessed: 2025-04-22. [Online]. Available: <https://contextily.readthedocs.io/en/latest/>
- [48] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. F. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, and T. Graepel, “Value-decomposition networks for cooperative multi-agent learning based on team reward,” in *Adaptive Agents and Multi-Agent Systems*, 2018.